GM

[EDICIÓN 2026]

Sistema de Bases de Datos

[GUÍA RÁPIDA DE ESTUDIO]

@garydevelop

Índice

Parte 1 · Modelo conceptual de datos **4**

1 Introducción a los sistemas de bases de datos 4
2 Entidades y atributos 5
3 Relaciones entre entidades 6
4 Restricciones y notaciones del diagrama 7

Parte 2 · Bases de datos relacionales **8**

5 Tablas, filas y columnas 8
6 Claves primarias y foráneas 9
7 Dependencias funcionales y cierre 10
8 Formas normales y descomposición 11
9 Del diagrama E-R al esquema relacional 12

Parte 3 · Álgebra relacional **13**

10 Operaciones básicas: selección, proyección, renombrar y producto 13
11 Operadores de conjuntos: unión, intersección y diferencia 14
12 Operaciones derivadas: reuniones (joins) y semirreunión 14
13 División y consultas «para todo» 15
14 Agregación, agrupamiento y optimización de consultas 16

Parte 4 · Lenguaje estructurado de consultas (SQL) **18**

15 De álgebra relacional a SQL: SELECT, renombrar, DISTINCT y ORDER BY 18
16 Objetos simples: CREATE, ALTER, DROP, tipos y restricciones 19
17 Manipular datos: INSERT, UPDATE, DELETE y operadores 20
18 Agregación, agrupamiento, reuniones y operaciones de conjuntos 21
19 Subconsultas y vistas 22

Parte 5 · Objetos y estructuras avanzadas **23**

20 Procedimientos almacenados y funciones 23
21 Disparadores (triggers) 23
22 Transacciones, ACID, aislamiento y bloqueos 24
23 Índices: árbol B+, hash y costo de una consulta 25

Parte 6 · Seguridad y conceptos avanzados **27**

24 Seguridad básica e inyección SQL 27
25 Administración de usuarios y privilegios 28
26 Bases de datos NoSQL: tipos, teorema CAP y BASE 29
27 Data Warehouse: OLAP, esquema en estrella y ETL 30

Parte 7 · Formulario general **32**

Cómo usar esta guía

Una guía de **Sistema de Bases de Datos** en lenguaje natural: cada regla o sentencia en su cuadro, con dónde se usa, para qué sirve y por qué funciona, y con ejemplos de matrículas, tiendas y bibliotecas. Al final está el formulario general.

1 · Lee la idea

Dos líneas que explican de qué trata el tema, en palabras simples.

2 · Memoriza el truco

La caja **RECUERDA** trae una comparación fácil para no olvidar la idea.

3 · Usa el cuadro

Cada fila es un formulario: **regla o sentencia, dónde se usa, para qué sirve y por qué funciona**.

Cómo leer una fila

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`SELECT col FROM tabla WHERE cond`	Consultar solo los estudiantes de una carrera o con promedio mayor a 8.	Devuelve las filas de la tabla que cumplen la condición y solo las columnas que pides.	Es la selección y la proyección del álgebra relacional dichas en inglés: primero se filtran las filas y después se eligen las columnas.

Sobre el curso

- Sistema de Bases de Datos (TICG1018) es un curso de formación profesional de la Facultad de Ingeniería en Electricidad y Computación (FIEC) de ESPOL: se aprende a modelar los datos de una organización, a guardarlos en tablas bien diseñadas y a consultarlos con SQL.
- La guía sigue el orden del sílabo: modelo conceptual, bases de datos relacionales, álgebra relacional, SQL, objetos avanzados (procedimientos, disparadores, transacciones e índices) y seguridad, NoSQL y Data Warehouse. Texto base: *Database Systems: Design, Implementation, & Management* (Coronel y Morris, 13.ª ed.).

Mapa de la guía

PARTE 1 **Modelo conceptual de datos** pág. 4
Del problema del mundo real al diagrama entidad-relación. (4 temas)

PARTE 2 **Bases de datos relacionales** pág. 8
Tablas, claves y normalización: cómo se guarda bien la información. (5 temas)

PARTE 3 **Álgebra relacional** pág. 13
El lenguaje formal de las consultas: seis operaciones y todo lo demás se arma con ellas. (5 temas)

PARTE 4 **Lenguaje estructurado de consultas (SQL)** pág. 18
Escribir en SQL lo que se razonó en álgebra relacional. (5 temas)

PARTE 5 **Objetos y estructuras avanzadas** pág. 23
Lo que vive dentro del servidor: procedimientos, disparadores, transacciones e índices. (4 temas)

PARTE 6 **Seguridad y conceptos avanzados** pág. 27
Quién puede hacer qué, y hacia dónde van las bases de datos modernas. (4 temas)

PARTE 7 **Formulario general** pág. 32
Lo más importante de la guía en una sola consulta, por parte y por tema. (211 reglas y sentencias)

Antes de empezar

- Una **tabla** tiene filas (registros) y columnas (atributos). La **clave primaria** identifica cada fila; la **clave foránea** apunta a la clave primaria de otra tabla.
- En álgebra relacional: σ elige filas, π elige columnas y $\bowtie$ une tablas. En SQL las palabras reservadas se escriben en mayúscula, por costumbre, no por obligación.
- Los ejemplos usan una base de ESPOL con estudiantes, materias y matrículas, para que reconozcas las tablas en cada tema.
- Los detalles pueden variar un poco entre Oracle, MySQL, PostgreSQL y SQL Server; la guía usa SQL estándar y avisa cuando algo cambia.

Material de estudio no oficial, hecho por estudiantes para estudiar. No reemplaza las clases, el sílabo ni el material oficial de la materia.

PARTE 1

Modelo conceptual de datos

Del problema del mundo real al diagrama entidad-relación.

TEMA 1

Introducción a los sistemas de bases de datos

Una base de datos es un conjunto organizado de datos relacionados, y el SGBD es el programa que la crea, la protege y la consulta. Se mira en tres niveles (externo, conceptual e interno) para que un cambio en uno no rompa los otros.

> **RECUERDA** Piensa en una biblioteca: los libros son los datos, el catálogo es el esquema y el bibliotecario es el SGBD, al que nadie se salta.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\text{datos} + \text{contexto} = \text{información}$	Un 85 suelto no dice nada; «Ana sacó 85 en Bases de Datos» sí.	Dato: valor crudo. Información: el dato con significado (quién, qué, cuándo) para tomar una decisión.	El dato es solo una pieza; al unirlo a su contexto ya responde una pregunta.
`SELECT * FROM ESTUDIANTE`	Una app de matrícula pide los estudiantes; nunca abre los archivos ella misma.	El SGBD (Oracle, MySQL, PostgreSQL o SQL Server) recibe la orden, la ejecuta y devuelve las filas.	Un solo programa vigila los datos: así no hay copias sueltas ni dos apps pisándose.
$\text{ESTUDIANTE}(\underline{matricula},\ nombre,\ carrera)$	Al diseñar: se escribe el nombre de la tabla y sus columnas; casi no cambia con el tiempo.	Esquema: la estructura. Esta tabla tiene 3 atributos y la clave primaria $matricula$ va subrayada.	Es el molde o plano: dice qué forma tienen los datos, sin guardar ninguno.
$\{(1001,\ \text{Ana},\ \text{Computación}),\ \ldots\}$	Lo que hay guardado hoy en la tabla; cambia con cada matrícula nueva.	Instancia: el conjunto de filas en un momento dado. Hoy ESTUDIANTE tiene 4 filas.	Es la foto de la tabla hoy: el molde es el mismo, pero las filas entran y salen.
$\text{externo} \leftrightarrow \text{conceptual} \leftrightarrow \text{interno}$	Para ordenar quién ve qué: usuarios, diseñador de la base y almacenamiento.	Externo: vistas por usuario. Conceptual: todas las tablas. Interno: archivos, páginas e índices.	Cada nivel oculta los detalles del de abajo, como en un restaurante: menú, cocina y bodega.
`CREATE VIEW v1 AS SELECT nombre FROM ESTUDIANTE`	Dar a cada usuario solo lo que necesita, como la lista de nombres de un curso.	Una vista es una consulta guardada con nombre. v_1 muestra solo $nombre$ y esconde la carrera.	Es una ventana a la tabla: se mira por ella y lo que queda fuera del marco no se ve.
`ALTER TABLE ESTUDIANTE ADD correo VARCHAR(60)`	Agregar una columna sin romper los reportes ni las apps que ya existen.	Independencia lógica: cambia el nivel conceptual y las vistas y programas antiguos siguen igual.	Las apps usan la vista, no la tabla; mientras la vista exista, nada se rompe.
`CREATE INDEX idx_nombre ON ESTUDIANTE(nombre)`	Acelerar las búsquedas por nombre sin tocar ninguna consulta ni programa.	Independencia física: cambia cómo se guarda (índices, discos) y las consultas dan lo mismo.	Una consulta dice qué quieres, no cómo buscarlo; el SGBD elige el camino.

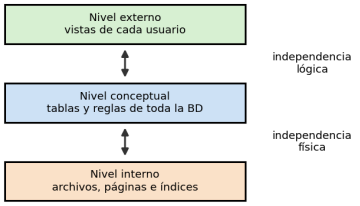
EJEMPLO RESUELTO

Esquema: $\text{ESTUDIANTE}(\underline{matricula}, nombre, carrera)$, grado 3. Instancia de hoy: 4 filas (Ana Loor, Luis Pazmiño, Marta Vera, Diego Ruiz). La vista v_1 devuelve los 4 nombres y oculta la carrera. Con ALTER TABLE el esquema pasa a 4 atributos, pero v_1 devuelve las mismas 4 filas (independencia lógica). Al crear el índice por nombre el resultado tampoco cambia, solo la rapidez (independencia física).

El libro base (Coronel y Morris) llama al SGBD «DBMS». Oracle usa VARCHAR2 en lugar de VARCHAR y escribe ADD (correo VARCHAR2(60)).

Gráficos de apoyo

Arquitectura de tres niveles



Los tres niveles

Cada flecha doble es una independencia: cambiar un nivel no obliga a cambiar el de arriba

Tabla ESTUDIANTE

Esquema: ESTUDIANTE(matricula, nombre, carrera)

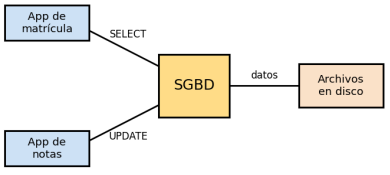
matricula	nombre	carrera
1001	Ana Loor	Computación
1002	Luis Pazmiño	Electrónica
1003	Marta Vera	Computación
1004	Diego Ruiz	Telemática

Instancia: las 4 filas de hoy

Esquema e instancia

Los títulos azules son el esquema; las 4 filas son la instancia de hoy

Aplicaciones, SGBD y archivos



El SGBD en el medio

Las aplicaciones nunca tocan los archivos: todo pasa por el SGBD

TEMA 2

Entidades y atributos

Una entidad es algo del que guardas datos, y sus atributos lo describen. Es fuerte si se identifica sola (ESTUDIANTE) y débil si necesita a otra para identificarse (AULA depende de EDIFICIO).

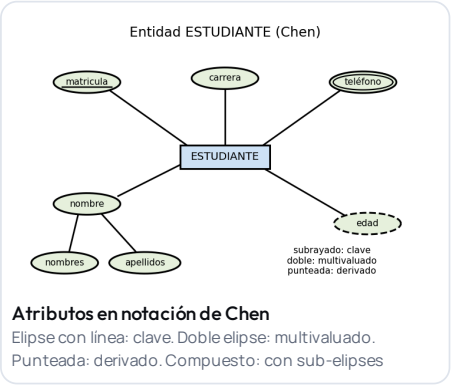
> **RECUERDA** La entidad es la ficha de un fichero, y los atributos son los casilleros de la ficha; la débil lleva escrito a quién pertenece.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\text{entidad} \to \text{tabla},\ \text{atributo} \to \text{columna}$	Al leer un enunciado: los sustantivos (estudiante, materia, profesor) suelen ser entidades.	Entidad: cosa sobre la que guardas datos. Atributo: una propiedad suya, como nombre o carrera.	Cada ocurrencia (Ana, Luis) será después una fila; cada atributo, una columna de esa tabla.
$nombre = (nombres,\ apellidos)$	Un dato que se parte en trozos con sentido, como el nombre completo.	Atributo compuesto: se divide en partes; las partes se guardan como columnas separadas.	Con nombres y apellidos aparte puedes ordenar o buscar por apellido sin trucos.
$telefono = \{t_1,\ t_2,\ \ldots\}$	Un estudiante con celular y teléfono fijo; una persona con varios correos.	Multivaluado: una sola entidad tiene varios valores del mismo atributo (t_1 , t_2).	Una celda no guarda listas; por eso cada valor pasa a su propia fila en otra tabla.
$edad = \lfloor (hoy - fnac)/365.25 \rfloor$	Edad, antigüedad o total a pagar: se calculan con otros datos.	Derivado: no se guarda, se calcula. Con $fnac$ = 14-may-2003, Ana tiene 23 años al 16-oct-2026.	Si guardaras la edad, quedaría vieja en cada cumpleaños; la fecha de nacimiento no cambia.
$\forall e_1 \neq e_2 :\ e_1[K] \neq e_2[K]$	Dos estudiantes nunca pueden tener la misma matrícula.	Identificador K (clave): sus valores no se repiten entre dos entidades distintas e_1 y e_2.	Con una clave repetida no sabrías a qué fila se refiere un borrar o un cambiar la nota.
$e[K] \neq \text{NULL}$	Al insertar: la base rechaza un estudiante sin matrícula.	La clave siempre debe tener valor. NULL significa «no se sabe» y no se puede identificar con un desconocido.	Es como una cédula en blanco: sin número no hay manera de encontrar a la persona.
$\text{AULA}(\underline{cod_edif^*,\ num_aula},\ capacidad)$	El aula 101 existe en varios edificios: el número solo no la identifica.	Entidad débil: su clave une la clave del dueño (cod_edif^*, foránea) y su parte propia num_aula.	«Aula 101» es ambiguo; «aula 101 del edificio 16B» ya es único.
`ON DELETE CASCADE`	Al borrar un edificio, sus aulas dejan de tener sentido y se borran con él.	La débil depende de la fuerte: sin el dueño no existe. Vale en Oracle, MySQL, PostgreSQL y SQL Server.	Un aula sin edificio no se puede ubicar: muere junto con quien la identifica.

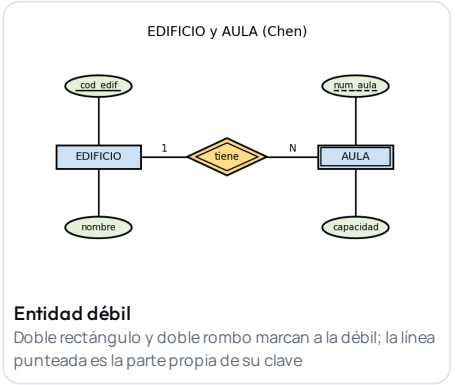
> **EJEMPLO RESUELTO**
> En $\text{AULA}(cod_edif^*, num_aula, capacidad)$ se guardan (16B, 101, 40), (11A, 101, 60) y (16B, 102, 35): el 101 se repite, pero cada pareja es única. Insertar (16B, 101, 50) falla por repetir la clave; (NULL, 103, 50), por clave nula; (99Z, 103, 50), porque el edificio 99Z no existe. Al borrar el edificio 16B, con CASCADE, desaparecen sus dos aulas y queda solo (11A, 101, 60).

En SQLite una clave de texto admite NULL si no pones NOT NULL; Oracle, MySQL, PostgreSQL y SQL Server siempre la rechazan.

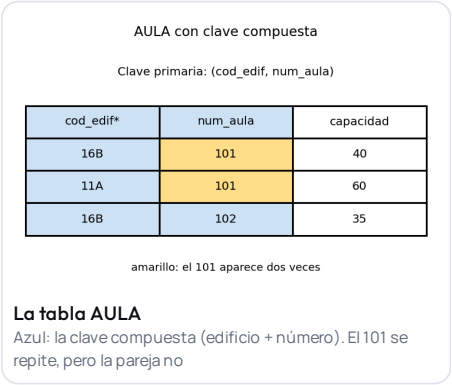
Gráficos de apoyo



Atributos en notación de Chen
Elipse con línea: clave. Doble elipse: multivaluado. Punteada: derivado. Compuesto: con sub-elipses



Entidad débil
Doble rectángulo y doble rombo marcan a la débil; la línea punteada es la parte propia de su clave



La tabla AULA
Azul: la clave compuesta (edificio + número). El 101 se repite, pero la pareja no

TEMA 3

Relaciones entre entidades

Una relación une entidades con un verbo (un estudiante se matricula en una materia). Se describe con su grado, su tipo (1:1, 1:N, M:N), su cardinalidad y si la participación es total o parcial.

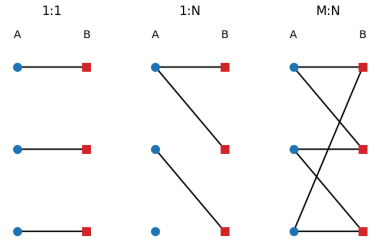
> **RECUERDA** Piensa en un baile: grado es cuántos bailan juntos, y el tipo dice si cada uno tiene una pareja fija, varias o ninguna.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\text{grado}(R) = n$	Contar cuántas entidades une el verbo: matricular une estudiante y materia.	Binaria: grado 2. Ternaria: grado 3. Recursiva: grado 2, con la misma entidad dos veces.	Es contar cuántos participantes tiene el verbo de la relación.
$\#\{\, b : (a, b) \in R \,\}$	Preguntar cuántas materias tiene Ana o cuántos estudiantes tiene TICG1018.	Cardinalidad: con cuántas ocurrencias b de B se relaciona una ocurrencia a de A.	Se cuenta desde un lado: en los datos de ejemplo Ana tiene 2 materias y Marta, 1.
$A\ (1{:}1)\ B$	Un profesor dirige un departamento y cada departamento tiene un solo director.	Cada a se une con a lo más un b y cada b con a lo más un a. Se asegura con UNIQUE.	Es una pareja exclusiva: nadie se repite en ninguno de los dos lados.
$A\ (1{:}N)\ B$	Un departamento tiene muchos profesores, pero cada profesor está en uno solo.	Un a se une con muchos b; cada b con un solo a. La clave foránea va en el lado «muchos».	Es un árbol de familia: un padre con varios hijos y cada hijo con un solo padre.
$A\ (M{:}N)\ B$	Estudiantes y materias: Ana cursa 2 materias y TICG1018 la cursan 2 estudiantes.	Muchos a con muchos b. Se resuelve con una tabla intermedia (MATRICULA) que lleva también la nota.	Una columna no guarda listas; la tabla intermedia tiene una fila por cada pareja.
$\text{total: } \forall a\, \exists b\, (a, b) \in R$	¿Todo estudiante debe tener matrícula (total) o puede no tenerla (parcial)?	Total: cada ocurrencia participa al menos una vez. Parcial: algunas quedan sin pareja.	De 4 estudiantes, 3 tienen matrícula y Diego no: la participación de ESTUDIANTE es parcial.
$\text{MATERIA} \xrightarrow{\text{requiere}} \text{MATERIA}$	Prerrequisitos: BD requiere Cálculo y Física A; Física A requiere Cálculo.	Recursiva: la misma entidad juega dos roles (la que requiere y la requerida). Grado 2.	Es una tabla que se mira en el espejo; se nombran los dos roles para no confundirlos.
$R(E_1,\ E_2,\ E_3)$	Profesor, materia y período: quién dicta qué y cuándo.	Ternaria (grado 3): une tres entidades a la vez. No siempre se reemplaza por tres binarias.	Cada fila es un trío; al partirlo en parejas se pierde qué trío exacto ocurrió.

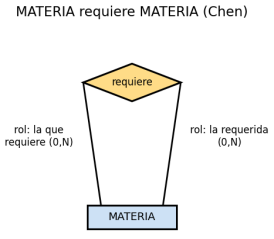
> **EJEMPLO RESUELTO**
> Ternaria PROFESOR–MATERIA–PERÍODO con 3 tríos: (Ana, BD, 2026-1), (Ana, Redes, 2026-2) y (Luis, BD, 2026-2). Si la partes en tres binarias (profesor–materia, materia–período, profesor–período) y las vuelves a unir, salen 4 tríos: aparece (Ana, BD, 2026-2), que nunca ocurrió. Por eso se conserva como una sola relación ternaria.

Un atributo que depende de la pareja completa (como la nota) pertenece a la relación M:N, no a ninguna de las dos entidades.

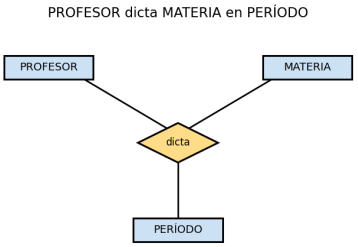
Gráficos de apoyo



Tipos de relación
Cada línea es una pareja: en 1:1 nadie repite, en 1:N repite el lado izquierdo y en M:N repiten los dos



Relación recursiva
La misma entidad MATERIA se une consigo misma con dos roles distintos



Relación ternaria
Un solo rombo une tres entidades a la vez: el trío (profesor, materia, período)

TEMA 4

Restricciones y notaciones del diagrama

Las restricciones son las reglas del negocio que el diagrama debe respetar. Se dibujan en notación de Chen o en pata de gallo, y cuando varias entidades comparten rasgos se agrupan con generalización y especialización.

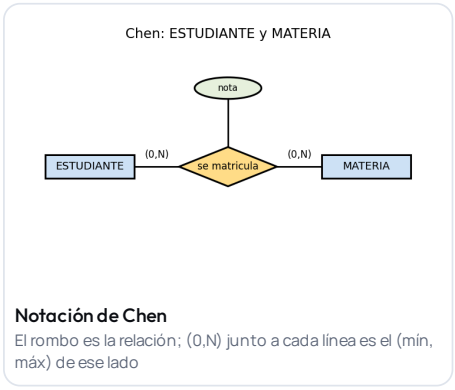
> **RECUERDA** Son las reglas de un juego: el diagrama es el tablero y las restricciones dicen qué jugadas valen y cuáles no.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$(0,N)$ $(1,1)$ $(1,N)$	ESTUDIANTE (0,N) MATRICULA: puede no tener materias o tener muchas.	Primer número (mínimo): 0 opcional, 1 obligatorio. Segundo (máximo): 1 una sola, N muchas.	Con dos números lees la participación y la cardinalidad a la vez.
`CHECK (nota BETWEEN 0 AND 100)`	Las notas en ESPOL van de 0 a 100; ninguna puede ser 105 ni negativa.	Dominio: valores permitidos de un atributo. La base rechaza el insert o update que se salga.	La base revisa cada valor al entrar, así ningún programa puede colar un dato absurdo.
`correo VARCHAR(60) UNIQUE`	Dos estudiantes no pueden compartir el mismo correo institucional.	UNIQUE impide repetidos en una columna que no es la clave. Admite NULL (en SQL Server, uno solo).	Es una segunda identificación: no manda en la tabla, pero tampoco se repite.
`nombre VARCHAR(40) NOT NULL`	Todo estudiante debe tener nombre; la carrera sí podría quedar en blanco.	NOT NULL obliga a dar un valor. Sin esa palabra, el atributo es opcional y acepta NULL.	Equivale al mínimo 1 del par (mín, máx); sin la regla entrarían filas incompletas.
$\square - \lozenge - \square$	Clases y apuntes: entidad = rectángulo, relación = rombo, atributo = elipse.	Notación de Chen: cada concepto tiene su figura y el par (mín, máx) va junto a la línea.	Es muy didáctica: ideal para pensar el problema antes de pasar a tablas.
$\mid = 1,$ $\circ = 0,$ $\prec = N$	MySQL Workbench y casi todas las herramientas de diseño dibujan en pata de gallo.	Pata de gallo: barra = uno, círculo = cero, pata = muchos. El (0,N) se dibuja como $\circ\!\prec$.	Las marcas se leen pegadas a la tabla: el círculo da el mínimo; la barra o la pata, el máximo.
$\text{PERSONA} \supset \{\text{ESTUDIANTE}, \text{PROFESOR}\}$	ESTUDIANTE y PROFESOR comparten cédula y nombre, pero cada uno tiene datos propios.	Generalización: sube lo común a un supertipo. Especialización: baja a subtipos con atributos propios.	Se lee «es un»: un estudiante es una persona y hereda sus atributos sin copiarlos.
(d) disjunta (o) solapada	Un ayudante de materia es estudiante y profesor a la vez: solapada.	Disjunta: un solo subtipo por persona; solapada: varios. Total: todos están en un subtipo; parcial: no.	Son dos preguntas: ¿puede estar en dos subtipos? y ¿puede quedar en ninguno?

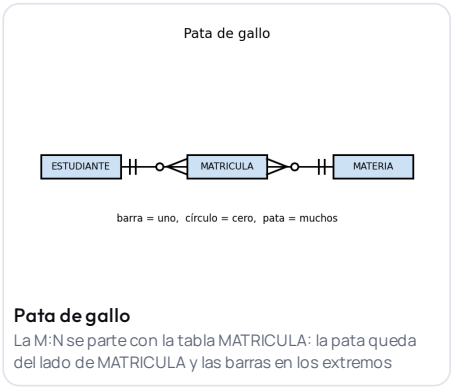
> **EJEMPLO RESUELTO**
> Tablas ESTUDIANTE(matricula, nombre NOT NULL, correo UNIQUE) y MATRICULA con CHECK de nota entre 0 y 100, con Ana ya guardada. Una nota de 105 falla (CHECK), un correo repetido falla (UNIQUE) y un nombre NULL falla (NOT NULL). En cambio, una nota de 85 y dos estudiantes sin correo (NULL) sí entran. Resultado: 3 estudiantes y 1 matrícula.

MySQL ignoraba CHECK antes de la versión 8.0.16. En pata de gallo una M:N se dibuja con una tabla intermedia (MATRICULA).

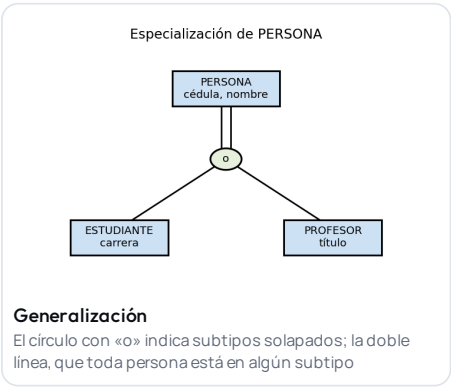
Gráficos de apoyo



Notación de Chen
El rombo es la relación; (0,N) junto a cada línea es el (mín, máx) de ese lado



Pata de gallo
La M:N se parte con la tabla MATRICULA: la pata queda del lado de MATRICULA y las barras en los extremos



Generalización
El círculo con «o» indica subtipos solapados; la doble línea, que toda persona está en algún subtipo

PARTE 2

Bases de datos relacionales

Tablas, claves y normalización: cómo se guarda bien la información.

TEMA 5

Tablas, filas y columnas

Una base de datos relacional guarda todo en tablas. Cada tabla es una relación: sus columnas son atributos, sus filas son tuplas y cada atributo toma valores de un dominio. Cuando falta un dato se usa NULL.

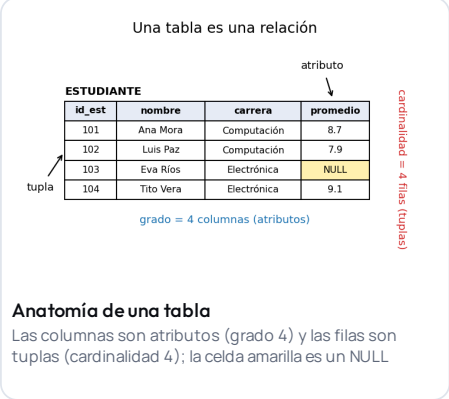
> **RECUERDA** Piensa en una hoja de cálculo muy ordenada: cada columna tiene un tipo de dato, cada fila es un registro y no puede haber dos filas idénticas.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$R(A_1, A_2, \ldots, A_n)$	Dar nombre y columnas a una tabla: ESTUDIANTE(id_est, nombre, carrera, promedio).	Esquema de la relación R: su nombre y la lista de atributos $A_1, \ldots, A_n$ (las columnas).	Es la plantilla de la tabla: dice qué columnas tiene, pero todavía no guarda ningún dato.
$t = \langle v_1, \ldots, v_n \rangle,\ v_i \in \mathrm{dom}(A_i)$	Una fila concreta, como el registro de Ana Mora en ESTUDIANTE.	Tupla t: un valor v_i por columna, y cada v_i sale del dominio de su atributo A_i.	El dominio es el conjunto de valores permitidos (texto, enteros, notas de 0 a 10): una columna no mezcla tipos.
$r \subseteq D_1 \times D_2 \times \cdots \times D_n$	Entender por qué una tabla no tiene filas repetidas ni un orden fijo.	La relación r (los datos guardados) es un subconjunto del producto de los dominios D_i.	Un conjunto no repite elementos ni tiene orden: dos filas iguales no existen y el orden de las filas no importa.
$\mathrm{grado}(R) = n$	Contar cuántas columnas tiene ESTUDIANTE: son 4.	Grado: cantidad de atributos n del esquema. Solo cambia si agregas o quitas columnas.	Es el ancho de la tabla: agregar filas no lo modifica.
$\mathrm{card}(r) = \lvert r \rvert$	Contar cuántos estudiantes hay guardados (4). No es la cardinalidad 1:N del diagrama E-R.	Cardinalidad de la relación: número de tuplas $\lvert r \rvert$. Cambia con cada INSERT o DELETE.	Es el alto de la tabla: crece cuando se guardan filas y baja cuando se borran.
`promedio IS NULL`	Buscar a quien aún no tiene promedio, como Eva Ríos, estudiante nueva.	Pregunta si el valor falta. NULL significa «desconocido o no aplica»; no es cero ni texto vacío.	Como no sabemos qué valor hay, no se puede comparar con $=$; por eso existe el operador especial IS NULL.
`NULL = NULL`	Escribir \texttt{promedio = NULL} por error: no devuelve ninguna fila, ni siquiera la de Eva.	El resultado no es verdadero ni falso sino desconocido (UNKNOWN), y WHERE solo deja pasar lo verdadero.	¿Un desconocido es igual a otro desconocido? No se sabe. Con tres valores de verdad la respuesta es «quizá».
`COUNT(promedio)`	Totales y promedios cuando hay NULL: COUNT(*) da 4 y COUNT(promedio) da 3.	Las funciones de agregación (COUNT de una columna, SUM, AVG) ignoran los NULL; COUNT(*) cuenta todas las filas.	Un valor desconocido no aporta al total: AVG divide entre las 3 notas reales, no entre las 4 filas.

> **EJEMPLO RESUELTO**
> ESTUDIANTE(id_est, nombre, carrera, promedio) tiene 4 filas: Ana 8.7, Luis 7.9, Eva sin promedio (NULL) y Tito 9.1. Grado 4, cardinalidad 4. `WHERE promedio` > 8 devuelve a Ana y Tito; `promedio` ≤ 8 devuelve a Luis; Eva no sale en ninguna ($2 + 1 = 3$ de 4 filas). `promedio` $=$ `NULL` no devuelve nada, pero IS NULL devuelve a Eva. AVG(promedio) = (8.7 + 7.9 + 9.1)/3 = 8.57.

En el modelo relacional «relación» es la tabla; no la confundas con la «relación» entre entidades del diagrama E-R. Una clave primaria (siguiente tema) es lo que evita las filas repetidas.

Gráficos de apoyo



Anatomía de una tabla
Las columnas son atributos (grado 4) y las filas son tuplas (cardinalidad 4); la celda amarilla es un NULL



El NULL y las condiciones
WHERE solo deja pasar las filas verdaderas (verde); a Eva, con promedio NULL, todas las condiciones le dan «desconocido»

TEMA 6

Claves primarias y foráneas

Una clave es un conjunto de atributos que identifica una fila. La clave primaria no se repite ni es NULL, y la clave foránea apunta a la primaria de otra tabla para que no queden datos huérfanos.

> **RECUERDA** La clave primaria es la cédula de la fila; la foránea es escribir la cédula de otra persona para referirte a ella.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$t_1 \neq t_2 \Rightarrow t_1[SK] \neq t_2[SK]$	Saber qué columnas de ESTUDIANTE identifican a alguien sin duda.	Superclave SK: conjunto de atributos que nunca se repite en dos filas distintas. Puede traer atributos de más.	Si algo ya es único, agregarle columnas lo sigue siendo: (id_est, nombre) también identifica.
$\nexists\, K' \subsetneq K :\ K' \text{ es superclave}$	Hallar las opciones reales para identificar: id_est o cédula, cada una sola.	Clave candidata K: superclave mínima; si le quitas cualquier atributo, deja de ser única.	Se quitan atributos hasta que ya no se pueda; lo que queda es lo mínimo que identifica.
`PRIMARY KEY` = `UNIQUE` + `NOT NULL`	Elegir id_est como clave de ESTUDIANTE; las otras candidatas (cédula) llevan UNIQUE.	Clave primaria: la candidata escogida. Es única y nunca NULL (integridad de entidad).	Se elige una sola para que todos señalen la fila igual; conviene corta y que casi no cambie.
$\pi_{FK}(R_1) \subseteq \pi_{PK}(R_2)$	Cada id_prof de MATERIA debe existir en PROFESOR (salvo que sea NULL).	Integridad referencial: los valores de la clave foránea FK de R_1 existen en la clave primaria PK de R_2.	Un dato que apunta a algo que no existe es un enlace roto; la regla evita materias con profesor fantasma.
`FOREIGN KEY (id_prof) REFERENCES PROFESOR`	Dentro de CREATE TABLE MATERIA. En SQLite hay que activar antes PRAGMA foreign_keys = ON.	Declara la clave foránea. Sin más acciones, borrar un profesor con materias se rechaza (NO ACTION).	El sistema revisa cada INSERT, UPDATE y DELETE contra la regla y rechaza lo que la rompe.
`ON DELETE CASCADE`	Al borrar un profesor también se borran sus materias; sirve con filas que no valen solas.	Acción al borrar: elimina en cadena todas las filas que apuntaban a la fila borrada.	Si el padre desaparece, los hijos pierden su razón de existir: el borrado se propaga como fichas de dominó.
`ON DELETE SET NULL`	Borrar un profesor pero conservar sus materias, que quedan sin profesor.	Acción al borrar: pone NULL en la clave foránea de los hijos. La columna debe aceptar NULL.	Se conserva el historial y se avisa con NULL que ya no hay profesor asignado.
`ON UPDATE CASCADE`	Cambiar el id_prof de un profesor sin romper las materias que lo usan.	Acción al actualizar: el nuevo valor de la clave se copia en los hijos. Oracle no la tiene.	La clave cambió en el padre y los hijos la siguen solos, en vez de apuntar al valor viejo.

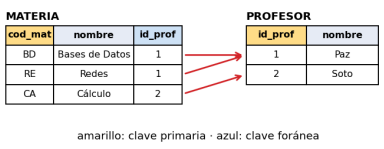
EJEMPLO RESUELTO

PROFESOR: 1 Paz, 2 Soto. MATERIA: BD (prof. 1), RE (prof. 1) y CA (prof. 2), con id_prof como clave foránea. Al borrar a Paz: con SET NULL, BD y RE quedan con id_prof NULL; con CASCADE se borran BD y RE y queda solo CA; con RESTRICT o NO ACTION hay error y no se borra nada. Insertar una materia con id_prof 99 también falla: ese profesor no existe.

En ESTUDIANTE(id_{est}, cédula, nombre, carrera), con id_{est} y cédula únicos, hay 12 superclaves y 2 candidatas. En MATRICULA la clave primaria es compuesta: (id_{est}, cod_{mat}, periodo).

Gráficos de apoyo

Clave foránea → clave primaria

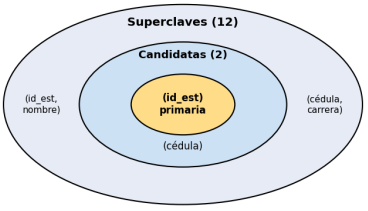


amarillo: clave primaria · azul: clave foránea

La foránea apunta a la primaria

Cada id_{prof} de MATERIA (azul) señala una fila de PROFESOR, cuya clave primaria es amarilla

ESTUDIANTE: tipos de clave



Superclaves, candidatas y primaria

Las claves candidatas son las superclaves mínimas, y la primaria es la candidata que escogemos

Borrar al padre (PROFESOR)

ON DELETE	Al borrar a Paz
CASCADE	se borran BD y RE
SET NULL	BD y RE quedan con id_prof NULL
RESTRICT / NO ACTION	error: no se borra a Paz

Sin acción explícita se usa NO ACTION

Acciones al borrar

Qué ocurre con las materias BD y RE cuando se borra al profesor Paz, según la acción elegida

TEMA 7

Dependencias funcionales y cierre

Una dependencia funcional $X \to Y$ dice que si dos filas tienen el mismo X, tienen el mismo Y. Los axiomas de Armstrong permiten deducir más dependencias, y el cierre X^+ dice qué atributos determina X.

> **RECUERDA** Es como la cédula: si conoces la cédula, el nombre queda determinado. El cierre es todo lo que puedes averiguar a partir de una pista.

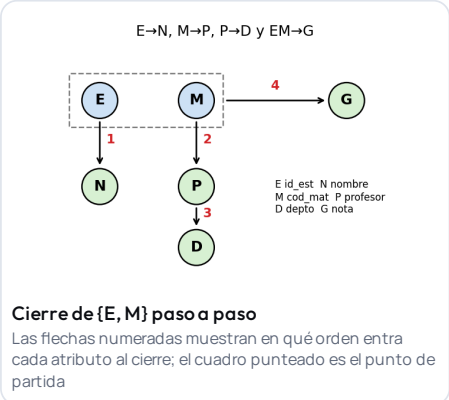
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$X \to Y$	Cada id_est (E) tiene un solo nombre (N): E → N. Cada materia (M) tiene un profesor (P): M → P.	Dependencia funcional: X determina a Y. Si dos filas coinciden en X, también coinciden en Y.	X funciona como una llave que abre un único valor de Y. Sale de las reglas del negocio, no de mirar datos.
$Y \subseteq X \Rightarrow X \to Y$	Decir que (E, M) determina a E: es obvio, es una dependencia trivial.	Reflexividad: un conjunto siempre determina a cualquiera de sus propios atributos.	Si ya conoces X, conoces sus partes. No da información nueva, pero es el punto de partida.
$X \to Y \Rightarrow XZ \to YZ$	Sumar el mismo atributo a ambos lados: de E → N sale EM → NM.	Aumento: añadir Z a los dos lados de una dependencia la deja cierta.	Si X ya fija a Y, agregar datos extra a la izquierda no la rompe; lo extra se copia a la derecha.
$X \to Y,\ Y \to Z \Rightarrow X \to Z$	Seguir una cadena: M → P y P → D (depto.) implican M → D.	Transitividad: si X determina a Y y Y determina a Z, entonces X determina a Z.	Es seguir pistas: la materia te lleva al profesor y el profesor te lleva al departamento.
$X \to Y,\ X \to Z \Rightarrow X \to YZ$	Juntar o separar lados derechos: M → P y M → D es lo mismo que M → PD.	Unión (y su inversa, descomposición): X determina a YZ si y solo si determina a Y y a Z.	Se deduce de los tres axiomas. Sirve para escribir cada dependencia con un solo atributo a la derecha.
$X^+ \leftarrow X^+ \cup Z \quad (Y \subseteq X^+,\ Y \to Z)$	Repetir la regla hasta que no entre nada nuevo: es el algoritmo del cierre de atributos.	Cierre X^+: empieza con X; si F tiene $Y \to Z$ y Y ya está dentro, agrega Z. Para cuando nada cambia.	Cada paso es una transitividad: lo que ya conoces sirve de pista para conocer más.
$X^+ = R \iff X$ superclave	Comprobar si (E, M) sirve de clave de la tabla grande: su cierre es todo R.	Si el cierre trae todos los atributos de R, X identifica cada fila: es superclave.	Conocer X permite deducir el resto de la fila; dos filas con igual X tendrían que ser la misma.
$X^+ = R,\ \ (X - A)^+ \neq R\ \ \forall A \in X$	Buscar la clave: probar atributos o parejas hasta hallar una que sea mínima.	Clave candidata: su cierre es R y, al quitarle cualquier atributo A, el cierre ya no es R.	Es la superclave sin sobrantes. Probar primero los atributos que nunca están a la derecha de una flecha.

EJEMPLO RESUELTO

$R(E, N, M, P, D, G)$: E = id_est, N = nombre, M = cod_mat, P = profesor, D = departamento, G = nota. $F = \{E \to N,\ M \to P,\ P \to D,\ EM \to G\}$. Cierre de $\{E, M\}$: parte de $\{E, M\}$; $E \to N$ agrega N; $M \to P$ agrega P; $P \to D$ agrega D; $EM \to G$ agrega G. Queda $\{E, N, M, P, D, G\} = R$, así que EM es superclave. Como $E^+ = \{E, N\}$ y $M^+ = \{M, P, D\}$ no bastan, EM es la única clave candidata.

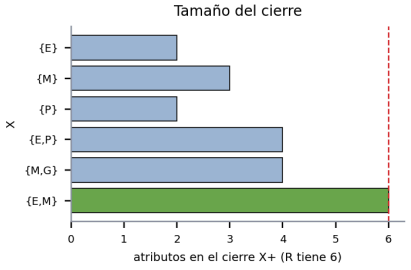
Atajo: los atributos que nunca aparecen a la derecha de ninguna flecha (aquí E y M) están en toda clave. Con este F hay 16 superclaves: todas las que contienen a E y M.

Gráficos de apoyo



Cierre de {E, M} paso a paso

Las flechas numeradas muestran en qué orden entra cada atributo al cierre; el cuadro punteado es el punto de partida



Quién alcanza todos los atributos

Tamaño del cierre de varios conjuntos: solo {E, M} llega a los 6 atributos de R

TEMA 8

Formas normales y descomposición

La normalización parte una tabla con datos repetidos en tablas más pequeñas, guiándose por las dependencias funcionales. Cada forma normal (1FN, 2FN, 3FN, FNBC) pide más que la anterior y evita anomalías de inserción, borrado y actualización.

> **RECUERDA** Es ordenar el clóset: cada cosa en su cajón, para que cambiar o botar una prenda no desordene todo lo demás.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\forall A \in R:\ t[A]$ es un solo valor	Una celda con dos teléfonos «0991, 0992»: hay que separarlos en filas o en otra tabla.	1FN: cada celda guarda un valor atómico; no hay listas ni columnas repetidas (tel1, tel2).	Para buscar, ordenar y comparar, el sistema necesita un valor por celda; una lista dentro de una celda no se filtra bien.
$X \subsetneq K \Rightarrow X \nrightarrow A$	En GENERAL, M → P depende solo de una parte de la clave (id_est, cod_mat).	2FN: en 1FN y ningún atributo no primo A depende de parte de la clave K (no primo: fuera de toda clave).	Si una parte de la clave basta para fijar A, ese dato se repite en cada fila donde aparece el resto de la clave.
$X \to A \Rightarrow X$ superclave $\vee$ A primo	Revisar tablas como MATERIA(cod_mat, prof, depto), donde depto depende de prof.	3FN: toda dependencia no trivial parte de una superclave o termina en un atributo primo (de alguna clave).	Elimina la dependencia transitiva: un atributo común depende de la clave solo a través de otro atributo.
$X \to A \Rightarrow X$ es superclave	Casos raros: estudiante, materia y profesor, donde cada profesor da una sola materia (P → M).	FNBC: todo determinante X de una dependencia no trivial es superclave. Es más estricta que la 3FN.	Quien decide el valor de otros atributos debe poder identificar la fila; si no, hay repetición.
$R_1 \cap R_2 \to R_1 \ \vee\ R_1 \cap R_2 \to R_2$	Antes de partir GENERALen tablas: comprobar que no se pierde información.	Descomposición sin pérdida: lo común de R_1 y R_2 es clave de al menos una de las dos.	Así cada fila de una se une con exactamente una de la otra y la reunión no inventa filas falsas.
$R = R_1 \bowtie R_2 \bowtie \cdots \bowtie R_k$	Recuperar los datos originales uniendo ESTUDIANTE, MATERIA, PROFESOR y MATRICULA.	Reunión natural de las partes: si la descomposición es sin pérdida, da exactamente la tabla original R.	Partir y volver a pegar por las columnas comunes devuelve lo mismo, sin filas de más ni de menos.
$(F_1 \cup F_2 \cup \cdots)^+ = F^+$	Mantener las reglas del negocio verificables dentro de las tablas nuevas.	Preservación de dependencias: las dependencias de las partes bastan para deducir todas las de R.	Si una regla queda repartida entre dos tablas, vigilarla obligaría a unirlas en cada INSERT.
`UPDATE GENERAL SET depto = 'X' WHERE prof = 'Paz'`	Cambiar el departamento de Paz en la tabla GENERAL, donde aparece en varias filas.	Anomalía de actualización: hay que cambiar todas las filas con ese dato; si olvidas una, queda inconsistente.	El mismo hecho está guardado varias veces. Con tablas separadas se cambia en un solo lugar.

> **EJEMPLO RESUELTO**
> GENERAL(id_est, nombre, cod_mat, prof, depto, nota), clave (id_est, cod_mat), tiene 6 filas y Paz-Computación aparece 3 veces: cambiar su departamento toca 3 filas (actualización). Si se borra la única matrícula de Redes se pierde a Vera y su depto. (borrado), y no se puede registrar una materia sin estudiantes (inserción). Con M → P → D, GENERAL es 1FN. Partida en ESTUDIANTE (3 filas), MATERIA (3), PROFESOR (3) y MATRICULA (6), la reunión devuelve las 6 filas.

Caso FNBC: $R(E, M, P)$ con $EM \to P$ y $P \to M$ (cada profesor da una sola materia). Claves: EM y EP. Es 3FN (M es primo) pero no FNBC (P no es superclave). Partir en (P, M) y (E, P) no pierde filas, pero ya no se puede vigilar $EM \to P$.

Gráficos de apoyo

Anomalías por datos repetidos

GENERAL

id_est	nombre	cod_mat	prof	depto	nota
101	Ana	BD	Paz	Computación	9.0
101	Ana	CA	Soto	Matemáticas	8.0
102	Luis	BD	Paz	Computación	7.5
102	Luis	RE	Vera	Computación	6.0
103	Eva	BD	Paz	Computación	8.5
103	Eva	CA	Soto	Matemáticas	9.5

naranja: se repite 3 veces (actualización)
rojo: único lugar donde está Vera (borrado)

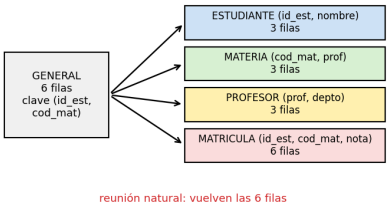
Una tabla con datos repetidos
En GENERAL, Paz y su departamento se repiten 3 veces (naranja) y Vera aparece en una sola fila (rojo)

1FN > 2FN > 3FN > FNBC



Cada forma normal pide más
Las formas normales están una dentro de otra: toda tabla en FNBC también cumple la 3FN, la 2FN y la 1FN

Descomposición sin pérdida



Partir GENERAL en cuatro tablas
La descomposición sin pérdida reparte los datos sin repetir; la reunión natural recupera las 6 filas originales

TEMA 9

Del diagrama E-R al esquema relacional

Cada pieza del diagrama E-R tiene una regla de conversión a tablas. Las entidades se vuelven tablas, la 1:N pone una clave foránea, y la M:N y los atributos multivaluados necesitan una tabla nueva.

> **RECUERDA** Es traducir un plano a materiales: cada caja del diagrama es una tabla y cada línea se vuelve una columna que apunta a otra tabla.

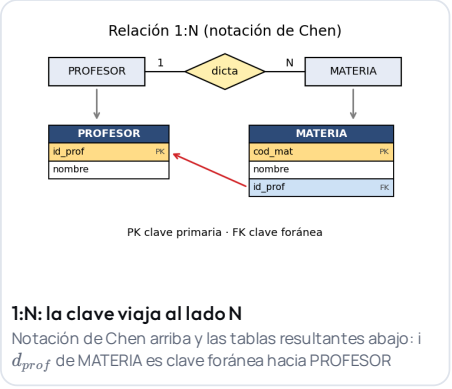
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
ESTUDIANTE($\underline{id_est}$, $nombre$, $carrera$)	Cada rectángulo del diagrama E-R: ESTUDIANTE, PROFESOR, MATERIA.	Entidad fuerte: una tabla con sus atributos como columnas. Subrayado = clave primaria.	Cada entidad es un grupo de cosas del mismo tipo: cada cosa es una fila y su identificador es la clave.
MATERIA($\underline{cod_mat}$, $nombre$, id_prof^*)	PROFESOR dicta muchas MATERIA y cada materia tiene un profesor (1:N).	Relación 1:N: la clave del lado «1» se copia como clave foránea (asterisco) en la tabla del lado «N».	El lado N tiene muchas filas y cada una apunta a un solo padre; al revés habría listas dentro de una celda.
MATRICULA($\underline{id_est^*, cod_mat^*, periodo}$, $nota$)	Un estudiante cursa muchas materias y una materia tiene muchos estudiantes (M:N).	Relación M:N: tabla nueva con las dos claves foráneas, que juntas forman la clave primaria. La nota va aquí.	Cada fila es una pareja concreta (Ana cursó BD en 2026-1); sin esta tabla habría listas en una celda.
OFICINA($\underline{cod_ofi}$, id_prof^* UNIQUE)	PROFESOR tiene una OFICINA y cada oficina es de un solo profesor (1:1).	Relación 1:1: la clave foránea va en una tabla y se declara UNIQUE; conviene en el lado de participación total.	UNIQUE impide que dos oficinas apunten al mismo profesor, y eso convierte el 1:N en 1:1.
PARALELO($\underline{cod_mat^*, numero}$, $cupo$)	El paralelo 1 de BD solo existe dentro de una materia: entidad débil.	Entidad débil: su clave primaria junta la clave de la entidad dueña (foránea) y su discriminante (numero).	Por sí solo «paralelo 1» no identifica nada; con la materia sí. Si se borra la materia, se borran sus paralelos.
TELEFONO($\underline{id_est^*, telefono}$)	Un estudiante puede tener varios teléfonos (atributo multivaluado).	Atributo multivaluado: tabla nueva con la clave del dueño (foránea) y el valor; ambas forman la clave.	Una celda no guarda listas (1FN); una fila por teléfono sí, y caben 1, 2 o 10.
BECADO($\underline{id_est^*}$, $porcentaje$)	ESTUDIANTE se especializa en BECADO y AYUDANTE (generalización).	Una tabla por superclase y una por subclase; la clave de la subclase es primaria y foránea a la vez.	Cada becado tiene una fila en ESTUDIANTE y otra en BECADO con el mismo id_est. Sin NULL de sobra.
ESTUDIANTE($\underline{id_est}$, $tipo$, $porcentaje$, $\ldots$)	Subclases con pocos atributos propios y consultas que casi siempre piden todo junto.	Una sola tabla con un atributo «tipo»; las columnas de las subclases quedan en NULL cuando no aplican.	Evita reuniones, pero llena de NULL y no permite exigir NOT NULL en columnas propias de una subclase.

EJEMPLO RESUELTO

Un diagrama tiene 4 entidades (ESTUDIANTE, PROFESOR, MATERIA y PARALELO, débil), el teléfono multivaluado, PROFESOR–MATERIA 1:N y ESTUDIANTE–MATERIA M:N. Salen 6 tablas: las 4 entidades, TELEFONO y MATRICULA. La 1:N no crea tabla, solo agrega id_prof a MATERIA. Al borrar BD con ON DELETE CASCADE desaparecen sus 2 paralelos y queda 1 (el de CA); un segundo paralelo 1 de BD se rechaza por la clave compuesta.

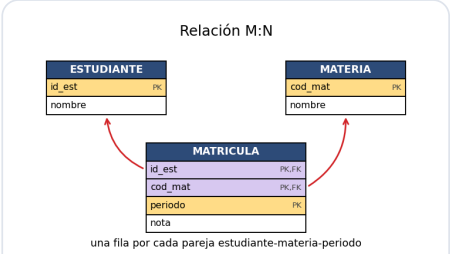
Notación: subrayado = clave primaria, asterisco = clave foránea. Un atributo compuesto (dirección) se parte en columnas (calle, ciudad); uno derivado (edad) no se guarda, se calcula. Una relación ternaria también lleva tabla nueva, con tres claves foráneas.

Gráficos de apoyo



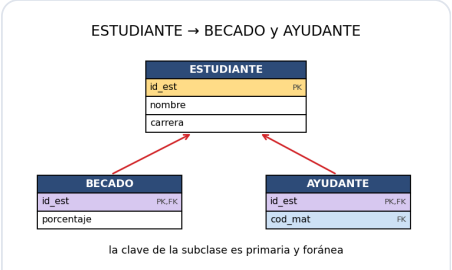
1:N: la clave viaja al lado N

Notación de Chen arriba y las tablas resultantes abajo: i d_{prof} de MATERIA es clave foránea hacia PROFESOR



M:N: la tabla puente

La relación M:N se convierte en una tabla nueva cuya clave primaria junta las dos claves foráneas y el periodo



Generalización: tabla por clase

Cada subclase tiene su tabla; su clave primaria es a la vez foránea hacia la tabla de la superclase

PARTE 3

Álgebra relacional

El lenguaje formal de las consultas: seis operaciones y todo lo demás se arma con ellas.

TEMA 10

Operaciones básicas: selección, proyección, renombrar y producto

En álgebra relacional cada operación recibe una o dos tablas y devuelve otra tabla. La selección σ filtra filas, la proyección π elige columnas, ρ cambia nombres y $\times$ combina todas las filas con todas.

RECUERDA Es una hoja de cálculo: σ oculta filas, π oculta columnas, ρ cambia los encabezados y $\times$ junta cada fila con cada fila.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\sigma_{carrera=\text{Computación}}(E)$	Listar solo a los estudiantes de una carrera, como pide Secretaría.	Devuelve las filas de E (ESTUDIANTE) donde la condición es verdadera. Salen Ana y Luis; mismas columnas.	Es un filtro fila por fila: cada fila pasa si cumple la condición y se descarta si no.
$\sigma_{prom\geq 8 \,\wedge\, carrera=\text{Electrónica}}(E)$	Combinar condiciones con y ($\wedge$), o ($\vee$) y no ($\neg$), como en una beca.	Solo quedan las filas que cumplen ambas. Se compara con $=$, $\neq$, $<$, $\leq$, $>$, $\geq$. Sale solo Marta.	Cada condición es verdadera o falsa por fila, como en lógica; $\wedge$ exige las dos y $\vee$ basta con una.
$\pi_{nombre,\, carrera}(E)$	Mostrar solo las columnas que necesita un reporte, sin id ni promedio.	Devuelve una tabla con solo esas columnas y elimina las filas repetidas. Quedan las 4 filas.	Una tabla es un conjunto de filas: si dos quedan idénticas al quitar columnas, cuentan como una sola.
$\pi_{carrera}(E)$	Averiguar qué carreras tienen estudiantes, sin repetir.	De 4 filas salen 2: Computación y Electrónica. SQL conserva repetidos salvo que escribas `DISTINCT`.	El álgebra clásica trabaja con conjuntos; SQL trabaja con bolsas y por eso necesita `DISTINCT`.
$\pi_{nombre}(\sigma_{prom>8}(E))$	Encadenar: primero filtrar filas y luego elegir columnas, como la lista de honor.	Se lee de adentro hacia afuera: filtra prom mayor a 8 y deja solo el nombre. Resultado: Ana y Marta.	Cada operación devuelve una tabla, así que su salida puede ser entrada de otra y se pueden anidar.
$\rho_X(E)$	Dar un alias corto a una tabla o usar la misma tabla dos veces en una consulta.	X es otro nombre de la misma tabla; las filas no cambian. SQL: `ESTUDIANTE X` (Oracle no admite AS).	Solo cambia la etiqueta, no los datos; sirve para evitar choques de nombres y distinguir dos copias.
$\rho_{S(codigo,\, alumno)}(R)$	Cambiar los nombres de las columnas antes de unir tablas o de mostrar un reporte.	Crea S con los mismos datos de R y las columnas llamadas codigo y alumno, en ese orden.	Las columnas se renombran por posición: la primera pasa a llamarse codigo y la segunda alumno. En SQL: `AS` .
$E \times M$	Probar todas las combinaciones estudiante-materia, antes de filtrar las que valen.	Cada fila de E con cada una de M: $4 \times 4 = 16$ filas y $4 + 3 = 7$ columnas. SQL: `CROSS JOIN`.	Es la tabla de multiplicar: cada elemento de un lado se empareja con todos los del otro, sin condición.

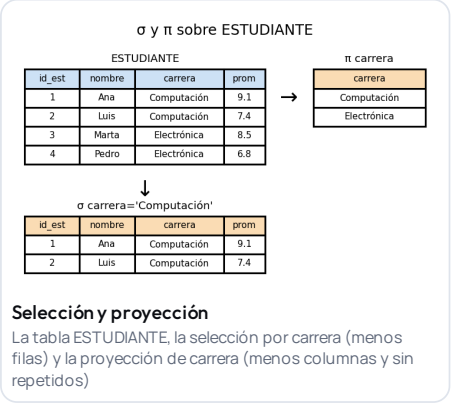
EJEMPLO RESUELTO

Nombre de los de Computación con promedio mayor a 8: $\pi_{nombre}(\sigma_{carrera=\text{Computación} \,\wedge\, prom>8}(E))$. 1) De las 4 filas, la carrera deja a Ana (9.1) y Luis (7.4). 2) El promedio mayor a 8 deja solo a Ana. 3) Se proyecta el nombre: el resultado es una tabla con una sola fila, Ana. En SQL:

```
SELECT nombre FROM ESTUDIANTE WHERE carrera = 'Computación' AND prom > 8.
```

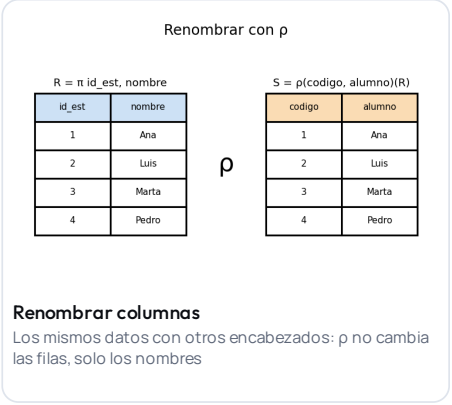
Abreviaturas del módulo: E = ESTUDIANTE(id_est, nombre, carrera, prom), M = MATERIA(cod, materia, cred) y N = MATRICULA(id_est, cod*, nota). El asterisco marca la clave foránea.*

Gráficos de apoyo



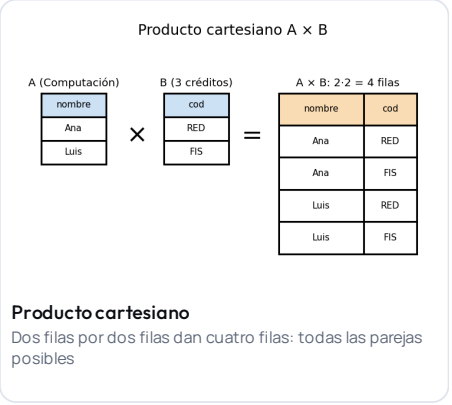
Selección y proyección

La tabla ESTUDIANTE, la selección por carrera (menos filas) y la proyección de carrera (menos columnas y sin repetidos)



Renombrar columnas

Los mismos datos con otros encabezados: ρ no cambia las filas, solo los nombres



Producto cartesiano

Dos filas por dos filas dan cuatro filas: todas las parejas posibles

TEMA 11

Operadores de conjuntos: unión, intersección y diferencia

Unión, intersección y diferencia tratan las tablas como conjuntos de filas. Solo se pueden usar entre tablas compatibles: mismo número de columnas y tipos parecidos, en el mismo orden.

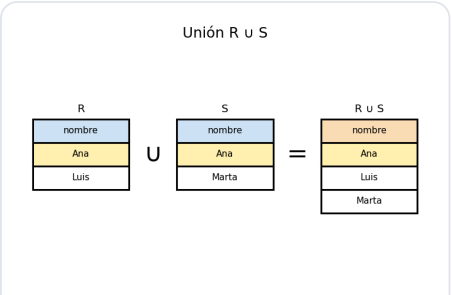
> **RECUERDA** Son listas de invitados: unión = todos los que salen en alguna lista, intersección = los de las dos, diferencia = los de una que no están en la otra.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$R \cup S$	Juntar dos listas, como los de Computación y los de promedio alto.	Filas de R, de S o de ambas, sin repetir. Con R = {Ana, Luis} y S = {Ana, Marta}: Ana, Luis y Marta.	Es juntar dos conjuntos: lo que se repite cuenta una sola vez, como en una lista de invitados.
$R \cap S$	Encontrar a quienes cumplen dos condiciones a la vez: Computación y promedio alto.	Filas que están en R y también en S. Con los mismos R y S solo queda Ana.	Una fila entra solo si pasa por las dos puertas; es el «y» de los conjuntos.
$R - S$	Quitar de una lista a los que ya están en otra: Computación sin los de promedio alto.	Filas de R que no están en S. Con R = {Ana, Luis} y S = {Ana, Marta} queda solo Luis.	Se parte de R y se tacha todo lo que aparece en S. Lo que solo está en S no entra.
$R - S \neq S - R$	Cuidar el orden al restar: Luis sale en $R - S$, pero Marta sale en $S - R$.	La diferencia no es conmutativa, a diferencia de la unión y la intersección.	No es lo mismo «mis amigos que no son tuyos» que «tus amigos que no son míos».
$R \cap S = R - (R - S)$	Armar la intersección con la diferencia; por eso no es una operación básica.	Quitar de R lo que no está en S deja justo lo que está en las dos. Resultado: Ana.	$R - S$ son los de R ajenos a S; al quitarlos de R solo quedan los comunes.
$\lvert R \cup S \rvert = \lvert R \rvert + \lvert S \rvert - \lvert R \cap S \rvert$	Contar las filas de una unión sin armarla: $2 + 2 - 1 = 3$.	$\lvert R \rvert$ es la cantidad de filas. Se resta la intersección para no contar dos veces a Ana.	Los comunes se contaron una vez en R y otra en S; se resta una para que valgan una sola.
$\mathrm{dom}(A_i) = \mathrm{dom}(B_i) \quad \forall i$	Antes de unir dos tablas: revisar que tengan igual número de columnas y tipos.	A_i y B_i son las columnas en la posición i de R y S. Si no coinciden, la operación no existe.	Una fila debe tener sentido: no se puede mezclar un nombre con una nota en la misma columna.

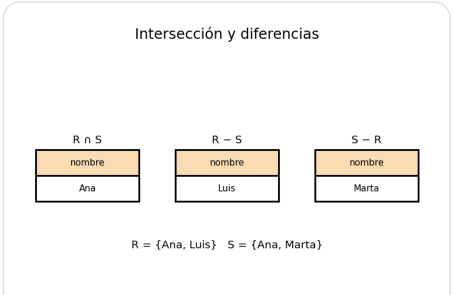
> **EJEMPLO RESUELTO**
> Sean $R = \pi_{nombre}(\sigma_{carrera=\text{Computación}}(E))$ = {Ana, Luis} y $S = \pi_{nombre}(\sigma_{prom \geq 8}(E))$ = {Ana, Marta}. Entonces $R \cup S$ = {Ana, Luis, Marta}, $R \cap S$ = {Ana}, $R - S$ = {Luis} y $S - R$ = {Marta}. Control: $2 + 2 - 1 = 3$ filas en la unión. Ambas tablas tienen una columna de texto, así que son compatibles.

En SQL: `UNION`, `INTERSECT` y `EXCEPT` (Oracle escribe `MINUS`; MySQL admite INTERSECT y EXCEPT desde la 8.0.31). `UNION ALL` conserva repetidos. Los nombres de columna salen de la primera tabla.

Gráficos de apoyo



Unión
R y S tienen a Ana en común; la unión la muestra una sola vez junto con Luis y Marta



Intersección y diferencias
Tres resultados distintos con las mismas R y S: lo común, lo que solo tiene R y lo que solo tiene S

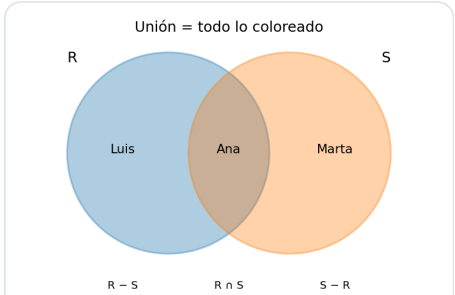


Diagrama de Venn de R y S
El centro es la intersección; a la izquierda queda R − S y a la derecha S − R

TEMA 12

Operaciones derivadas: reuniones (joins) y semirreunión

Una reunión (join) combina filas de dos tablas que se relacionan, como cada estudiante con sus matrículas. Es un producto seguido de una selección, pero se usa tanto que tiene su propio símbolo.

> **RECUERDA** Es pegar dos listas por un dato en común: la de estudiantes y la de matrículas, unidas por el número de estudiante.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$R \bowtie_\theta S = \sigma_\theta(R \times S)$	Unir con una condición cualquiera: la nota de una materia es menor que el promedio del estudiante.	Reunión theta: θ puede usar $=$, $\neq$, $<$, $>$. Con E y N (igual id_est y nota < prom) salen 5 filas.	Primero se hacen todas las parejas y luego se queda con las que cumplen θ: un atajo de dos pasos.

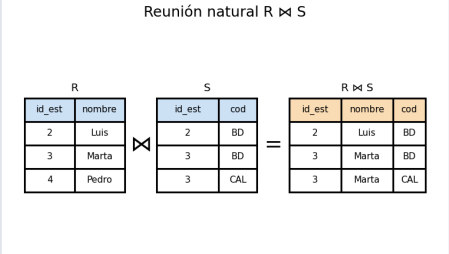
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$E \bowtie_{E.id_est = N.id_est} N$	Unir ESTUDIANTE con MATRICULA igualando id_est: cada matrícula con su estudiante.	Equirreunión: θ solo usa igualdades. Salen 6 filas y 7 columnas; id_est aparece repetido.	Es lo que hace una clave foránea: busca la fila cuya clave primaria es igual al valor guardado.
$E \bowtie N$	Unir tablas cuya columna de enlace se llama igual, como id_est en ESTUDIANTE y MATRICULA.	Reunión natural: iguala las columnas con el mismo nombre y deja una sola copia. Salen 6 filas y 6 columnas.	Es la equirreunión con la igualdad sobreentendida y la columna repetida borrada. En SQL: `NATURAL JOIN`.
$R \overset{\text{izq}}{\bowtie} S$	Listar a todos los estudiantes, incluso a quien no tiene matrículas, como Pedro.	Reunión natural más las filas de R sin pareja, rellenas con NULL. E con N: 7 filas; Pedro lleva NULL.	La reunión normal pierde las filas sin pareja; la externa las rescata. En SQL: `LEFT JOIN`.
$R \overset{\text{der}}{\bowtie} S$	Listar todas las materias, incluso las que nadie cursa, como Física.	Rescata las filas de S sin pareja. N con M: 7 filas; Física sale con NULL en id_est y nota.	Es la izquierda con los lados cambiados: mismo rescate, pero para la tabla de la derecha. `RIGHT JOIN`.
$R \overset{\text{comp}}{\bowtie} S$	Revisar qué le falta a cada lado en una conciliación de datos.	Natural más las filas sin pareja de ambos lados. E con N: 7 filas (solo Pedro queda sin pareja).	Suma el rescate de izquierda y de derecha. `FULL JOIN`; MySQL no lo trae y se arma con `UNION`.
$R \ltimes S = \pi_{\text{atr}(R)}(R \bowtie S)$	Quedarse solo con los estudiantes que tienen alguna matrícula, sin traer datos de ella.	Semirreunión: filas de R con al menos una pareja en S y solo columnas de R. Salen Ana, Luis y Marta.	Se une, pero se proyectan solo las columnas de R: funciona de filtro. En SQL: `IN` o `EXISTS`.
$\pi_{nombre,\ materia}(E \bowtie N \bowtie M)$	Mostrar a cada estudiante con el nombre de las materias en que está matriculado.	Se une de izquierda a derecha y se proyecta. Salen 6 filas, una por matrícula.	El resultado de la primera reunión es una tabla, así que se puede reunir con la tercera.

EJEMPLO RESUELTO

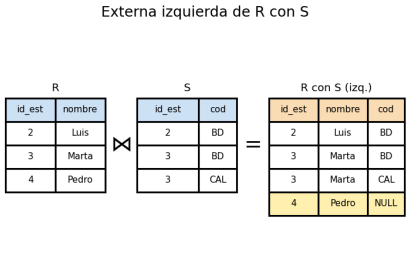
¿Qué materia cursa cada estudiante? Se reúne $E \bowtie N$ por id_est: salen 6 filas y Pedro desaparece porque no tiene matrículas. Luego $\bowtie M$ por cod agrega el nombre de la materia y π deja nombre y materia: 6 filas (Ana-Bases de Datos, Ana-Redes, Ana-Cálculo, Luis-Bases de Datos, Marta-Bases de Datos, Marta-Cálculo). Con reuniones externas izquierdas en vez de naturales aparece también Pedro con NULL: 7 filas.

Abreviaturas del módulo: E = ESTUDIANTE($\underline{id_est}$, nombre, carrera, prom), M = MATERIA($\underline{cod}$, materia, cred) y N = MATRICULA($\underline{id_est^*, cod^*}$, nota). El asterisco marca la clave foránea. La antirreunión (estudiantes sin matrícula) es $E - (E \ltimes N)$ y da a Pedro. Cuidado con `NATURAL JOIN`: si dos tablas comparten una columna por casualidad, la usa también; por eso MATERIA llama materia a su nombre.

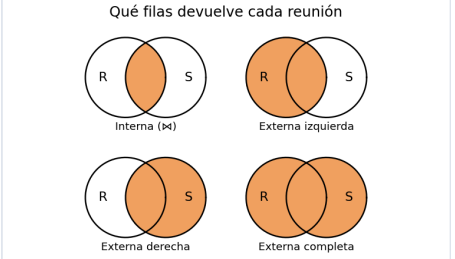
Gráficos de apoyo



Reunión natural
Cada fila de R se pega con las de S que tienen el mismo i d_{est}; Pedro no tiene pareja y desaparece



Reunión externa izquierda
Igual que la natural, pero Pedro se conserva con NULL en la columna que viene de S



Tipos de reunión en Venn
La zona pintada es lo que devuelve cada reunión: interna, izquierda, derecha o completa

TEMA 13

División y consultas «para todo»

La división responde preguntas con «para todo»: ¿qué estudiantes cursan todas las materias de un grupo? Devuelve los valores de R que aparecen emparejados con todas las filas de S.

RECUERDA Es el álbum de cromos: gana quien tiene todos los de la lista. Con uno solo que falte, queda fuera; si le sobran, no importa.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$R(A,B) \div S(B)$	Preguntas «para todo»: estudiantes que cursan todas las materias de 4 créditos.	Devuelve los a tales que (a,b) está en R para cada b de S. El resultado tiene solo la columna A.	Pregunta: ¿qué valor a aparece pegado a todos los b de la lista S?
$\pi_A(R) - \pi_A\big((\pi_A(R) \times S) - R\big)$	Armar la división con operaciones básicas; por eso es una operación derivada.	Se arman todos los pares candidato-divisor, se quitan los que existen en R, y los candidatos con faltantes salen.	Quien tiene un par faltante no cumple; al quitarlo de los candidatos quedan los que tienen todos.

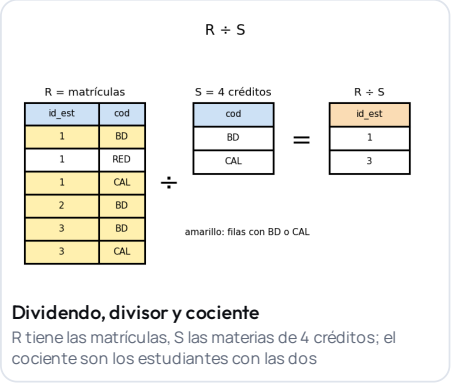
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\pi_{id_est,cod}(N) \div \pi_{cod}(\sigma_{cred=4}(M))$	Con N = MATRICULA y M = MATERIA: ¿quién cursa todas las materias de 4 créditos?	El divisor es {BD, CAL}. Ana y Marta tienen las dos; Luis solo BD. Cociente: id_est 1 y 3.	Ana tiene BD y CAL (y RED de sobra); Marta tiene BD y CAL; a Luis le falta CAL. Lo extra no molesta.
$\text{atr}(R \div S) = \text{atr}(R) - \text{atr}(S)$	Revisar que la división tenga sentido antes de escribirla.	Las columnas de S deben estar dentro de las de R; el resultado conserva las que sobran.	Se divide como en aritmética: lo que se quita al dividendo son las columnas del divisor.
$(R \div S) \times S \subseteq R$	Comprobar una división: reconstruir las parejas y ver que ya existen en R.	Cada a del cociente emparejado con todo S da pares que ya están en R (aquí 4 de 6).	Es la prueba de la división: cociente por divisor cabe dentro del dividendo, como $3 \times 2 \leq 7$.
$\pi_{id_est}(R \bowtie S)$	Cuando la pregunta es «alguna» materia y no «todas», no se divide: se reúne.	Estudiantes con al menos una materia de 4 créditos: Ana, Luis y Marta (3), frente a 2 con todas.	La reunión pide una pareja; la división las exige todas. «Existe» se hace con $\bowtie$ y «para todo» con $\div$.
$\pi_{id_est}(E) - (R \div S)$	Avisar a los estudiantes a quienes les falta alguna materia de 4 créditos.	Todos los estudiantes menos los que ya cumplen: quedan Luis y Pedro.	Se resta el cociente del total; Pedro aparece aunque no tenga matrículas porque el total sale de E.
`NOT EXISTS (... NOT EXISTS (...))`	SQL no tiene operador de división; se escribe con dos `NOT EXISTS` anidados.	Se lee: no existe materia de 4 créditos para la cual no exista una matrícula del estudiante.	Es la doble negación de «para todo»: todos cumplen cuando no existe alguien que no cumpla.

EJEMPLO RESUELTO

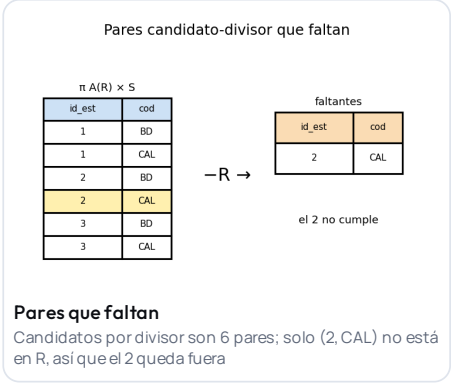
¿Quién cursa todas las materias de 4 créditos? Divisor: $\pi_{cod}(\sigma_{cred=4}(M))$ = {BD, CAL}. Dividendo: los 6 pares (id_est, cod) de MATRICULA. 1) Candidatos: los id_est que aparecen en el dividendo: 1, 2 y 3. 2) Pares candidato-divisor: 6; el único que no existe en el dividendo es (2, CAL), así que sale el 2. 3) Cociente = {1, 3}: Ana y Marta. En SQL también sirve `GROUP BY id_est HAVING COUNT(DISTINCT cod) = 2` sobre las materias de 4 créditos.

Si el divisor está vacío, todos cumplen «por vacío» y la fórmula devuelve todos los candidatos. El divisor puede ser una consulta: si sus filas cambian, cambia el cociente.

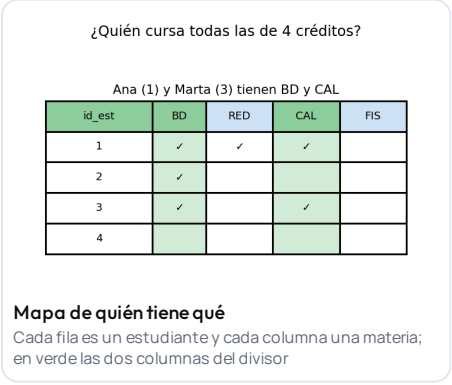
Gráficos de apoyo



Dividendo, divisor y cociente
R tiene las matrículas, S las materias de 4 créditos; el cociente son los estudiantes con las dos



Pares que faltan
Candidatos por divisor son 6 pares; solo (2, CAL) no está en R, así que el 2 queda fuera



Mapa de quién tiene qué
Cada fila es un estudiante y cada columna una materia; en verde las dos columnas del divisor

TEMA 14

Agregación, agrupamiento y optimización de consultas

La agregación γ resume muchas filas en una: cuenta, suma o promedia, y puede hacerlo por grupos. Optimizar es escribir la misma consulta de modo que se procesen menos filas, empujando las selecciones hacia abajo del árbol.

RECUERDA Es el cuadro de notas por curso: en vez de ver a cada estudiante, ves una línea por curso con la cantidad y el promedio.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\gamma_{\text{COUNT}(*),\ \text{MAX}(prom)}(E)$	Resumen de toda la tabla: cuántos estudiantes hay y el mejor promedio.	Sin grupos da una sola fila: 4 y 9.1. Otras: SUM, AVG, MIN.	Cada función recorre la columna y devuelve un único valor; sin grupos, toda la tabla es un solo grupo.
$_{carrera}\gamma_{\text{COUNT}(*),\ \text{AVG}(prom)}(E)$	Reporte por carrera: cantidad de estudiantes y promedio de cada una.	Una fila por carrera: Computación 2 y 8.25; Electrónica 2 y 7.65. Las columnas del grupo van primero.	Se forman grupos con el mismo valor de carrera y cada función se calcula dentro de su grupo. En SQL: `GROUP BY`.
$_{id_est}\gamma_{\text{COUNT}(cod),\ \text{AVG}(nota)}(N)$	Carga y rendimiento de cada estudiante: cuántas materias y qué nota promedio.	Ana: 3 y 9.0; Luis: 1 y 6.0; Marta: 2 y 7.5. Pedro no sale: no tiene filas en N.	Solo existen los grupos que tienen al menos una fila; sin matrículas no hay grupo.
$\sigma_{n \geq 2}\left({}_{id_est}\gamma_{\text{COUNT}(cod) \to n}(N)\right)$	Filtrar grupos, como estudiantes con 2 o más materias. En SQL es `HAVING`.	Una selección después de γ filtra grupos: quedan Ana (3) y Marta (2). Antes de γ filtra filas.	`WHERE` actúa sobre filas antes de agrupar y `HAVING` sobre los grupos ya calculados.

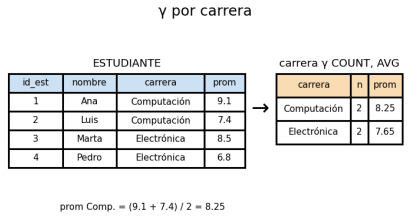
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\text{AVG}(x) = \dfrac{\text{SUM}(x)}{\text{COUNT}(x)}$	Promedio de notas: la suma entre la cantidad de notas, sin contar los NULL.	Con las 6 notas de N: 48 entre 6 da 8.0. $\text{COUNT}(*)$ cuenta filas; $\text{COUNT}(x)$, solo no nulas.	Las funciones saltan los NULL; si una nota vacía contara, el promedio saldría distorsionado.
$\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$	Separar una condición compuesta para empujar cada parte por separado hacia abajo.	Dos condiciones unidas con y equivalen a dos selecciones seguidas, en cualquier orden.	Una fila sobrevive si pasa los dos filtros; da igual cuál se aplique primero.
$\sigma_c(R \bowtie S) = \sigma_c(R) \bowtie S$	Filtrar antes de reunir cuando c solo mira columnas de R, para reunir menos filas.	Mismo resultado con menos trabajo: filtrar N con cod = BD antes de unirla con E.	Una fila que no pasa el filtro tampoco lo haría después; descartarla antes ahorra emparejarla en vano.
$\sigma_c(R \times S) = R \bowtie_c S$	Convertir un producto cartesiano más filtro en una reunión, que es la forma barata.	Con E (4 filas) y N (6) el producto tiene 24 filas; la reunión por id_est arma solo 6.	El producto arma todas las parejas aunque no sirvan; la reunión solo arma las que cumplen la condición.

EJEMPLO RESUELTO

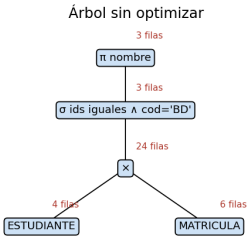
Nombres de quienes cursan BD. Plan ingenuo: $\pi_{nombre}(\sigma_{E.id_est=N.id_est \wedge cod=\text{BD}}(E \times N))$: el producto genera $4 \times 6 = 24$ filas y la selección deja 3. Plan optimizado: $\pi_{nombre}(E \bowtie \sigma_{cod=\text{BD}}(N))$: se filtra N a 3 filas y la reunión da 3. Ambos devuelven Ana, Luis y Marta, pero el segundo maneja 3 filas en vez de 24.

El motor de la base hace esta reescritura solo: estima el costo de varios planes y ejecuta el más barato. Para verlo: `EXPLAIN` *(MySQL, PostgreSQL),* `EXPLAIN PLAN FOR` *(Oracle) o el plan de ejecución gráfico de SQL Server. Como la reunión es conmutativa y asociativa, también elige el orden de las tablas.*

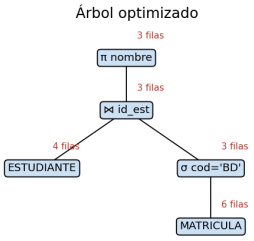
Gráficos de apoyo



Agrupar por carrera
Cuatro filas de ESTUDIANTE se resumen en dos: una por carrera, con la cantidad y el promedio



Árbol sin optimizar
Primero el producto arma 24 filas y recién arriba la selección las reduce a 3



Árbol optimizado
La selección baja hasta MATRICULA y la reunión solo trabaja con 3 filas

PARTE 4

Lenguaje estructurado de consultas (SQL)

Escribir en SQL lo que se razonó en álgebra relacional.

TEMA 15

De álgebra relacional a SQL: SELECT, renombrar, DISTINCT y ORDER BY

SQL dice lo mismo que el álgebra relacional, pero como una oración: SELECT elige columnas (π), FROM nombra la tabla y WHERE filtra filas (σ). Se lee casi como una frase en inglés.

> **RECUERDA** Es pedir en la cafetería: «de la carta (FROM), solo lo que cuesta menos de 5 USD (WHERE), y muéstrame nombre y precio (SELECT)».

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\pi_{\text{nombre}}(\sigma_{\text{edad}>20}(\text{ESTUDIANTE}))$	Pasar a SQL una consulta que ya razonaste en álgebra relacional.	Elige las filas con edad mayor que 20 (σ) y luego la columna nombre (π). En SQL: SELECT, FROM, WHERE.	SQL es el mismo álgebra con otra cara: π es SELECT, σ es WHERE y la relación va en FROM.
`SELECT nombre, carrera`	Cuando solo quieres algunas columnas: nombre y carrera, sin la edad.	Proyección π: lista las columnas que se muestran. Con SELECT * salen todas las columnas de la tabla.	Solo se queda lo que listas y en el orden en que lo escribes, igual que en π.
`FROM ESTUDIANTE`	Siempre: toda consulta dice de qué tabla saca los datos.	Nombra la relación de donde se leen las filas. Con dos tablas (FROM A, B) hace el producto cartesiano $\times$.	Es el punto de partida: se toma la tabla completa y después WHERE y SELECT recortan filas y columnas.
`WHERE edad > 20`	Quedarte solo con los mayores de 20 años, o con una carrera.	Selección σ: deja las filas donde la condición es verdadera. Las falsas o desconocidas se descartan.	Revisa fila por fila: si la condición da verdadero la fila pasa, si no se queda fuera.
`SELECT nombre AS alumno`	Ponerle un nombre más claro a una columna en un reporte.	Renombrar ρ: la columna sale como «alumno» en el resultado. La tabla guardada no cambia.	Es una etiqueta pegada al resultado, nada más. En casi todos los sistemas AS se puede omitir.
`FROM ESTUDIANTE E`	Para escribir E.nombre en vez de ESTUDIANTE.nombre, o usar la misma tabla dos veces.	Alias de tabla, $\rho_E(\text{ESTUDIANTE})$. Oracle no acepta AS entre la tabla y su alias.	Es un apodo corto válido solo dentro de esta consulta; evita confusiones cuando dos tablas comparten un nombre de columna.
`SELECT DISTINCT carrera`	Ver qué carreras hay, sin repetir una por cada estudiante.	Quita las filas repetidas: de 5 estudiantes salen 3 carreras distintas. Sin DISTINCT salen las 5.	El álgebra trabaja con conjuntos, que no repiten; SQL guarda los duplicados salvo que pidas DISTINCT.
`ORDER BY edad DESC, nombre`	Presentar la lista de mayor a menor edad y, si empatan, por nombre.	Ordena el resultado: ASC (de menor a mayor, por defecto) o DESC. Siempre va al final de la consulta.	Una tabla del álgebra no tiene orden; ORDER BY solo acomoda lo que se muestra, al terminar todo lo demás.

EJEMPLO RESUELTO

Nombre y carrera de los mayores de 20 años, del mayor al menor: SELECT nombre, carrera FROM ESTUDIANTE WHERE edad > 20 ORDER BY edad DESC. De los 5 estudiantes pasan Pedro (22) y Luis (21); salen (Pedro, Computacion) y (Luis, Computacion). Marta tiene 20 y no pasa, porque 20 > 20 es falso. SELECT carrera da 5 filas; con DISTINCT, solo 3.

Base de ejemplo de todo el módulo, con datos sin tildes para evitar problemas de codificación: ESTUDIANTE(id_est, nombre, carrera, edad), MATERIA(cod, nombre, creditos, id_prof*), PROFESOR(id_prof, nombre, depto) *y* MATRICULA(id_est*, cod*, nota). *El asterisco marca una clave foránea.*

Gráficos de apoyo

ESTUDIANTE (verde: edad > 20)

id_est	nombre	carrera	edad
1	Ana	Computacion	19
2	Luis	Computacion	21
3	Marta	Electronica	20
4	Pedro	Computacion	22
5	Sofia	Telematica	18

Filas y columnas que elige la consulta
Las filas verdes pasan el WHERE; luego SELECT se queda con nombre y carrera.

WHERE edad > 20 ORDER BY edad DESC

nombre	carrera
Pedro	Computacion
Luis	Computacion

Resultado de la consulta del ejemplo
Pedro y Luis, ya ordenados por edad de mayor a menor.

SELECT carrera

carrera
Computacion
Computacion
Electronica
Computacion
Telematica

SELECT DISTINCT carrera

carrera
Computacion
Electronica
Telematica

DISTINCT quita los repetidos
Sin DISTINCT salen 5 filas; con DISTINCT, 3 carreras distintas.

TEMA 16

Objetos simples: CREATE, ALTER, DROP, tipos y restricciones

CREATE TABLE crea una tabla con sus columnas, tipos y reglas. ALTER TABLE la modifica y DROP TABLE la borra con todos sus datos. Las restricciones hacen que la base rechace los datos malos antes de guardarlos.

> **RECUERDA** Las restricciones son el guardia de la puerta: revisa cada dato que quiere entrar y no deja pasar al que incumple las reglas.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`CREATE TABLE MATRICULA (...)`	Al armar la base: una tabla por cada entidad del diagrama.	Crea la tabla vacía MATRICULA. Entre paréntesis van las columnas con su tipo y las reglas, separadas por comas.	Es el plano de la tabla (su esquema): primero se define la forma y después se meten las filas.
`PRIMARY KEY (id_est, cod)`	Cuando una fila se identifica por una columna o por una pareja, como estudiante y materia.	Clave primaria: no admite repetidos ni NULL. Aquí la pareja (id_est, cod) no puede repetirse.	Repetir una matrícula sería contar dos veces al mismo estudiante en la misma materia; la clave lo impide.
`FOREIGN KEY (cod)` `REFERENCES MATERIA(cod)`	Para que una matrícula apunte solo a una materia que existe.	Clave foránea: el valor de cod debe existir en MATERIA, o la fila se rechaza. SQLite pide activarla (PRAGMA).	Es como anotar un número de cédula: debe corresponder a alguien registrado, o el vínculo queda roto.
`nombre VARCHAR(40) NOT NULL`	Datos obligatorios, como el nombre de cada estudiante.	VARCHAR(40) guarda texto de hasta 40 caracteres. NOT NULL prohíbe dejar la columna sin valor.	NULL significa «no se sabe»; NOT NULL obliga a dar un valor al insertar, así no hay estudiantes sin nombre.
`nombre VARCHAR(40) UNIQUE`	Valores que no deben repetirse sin ser la clave, como el nombre de una materia.	Rechaza una segunda fila con el mismo valor. Puede haber varias columnas UNIQUE, pero una sola clave primaria.	La base compara cada valor nuevo con los que ya están; si encuentra uno igual, no lo guarda.
`CHECK (nota BETWEEN 0 AND 10)`	Reglas del negocio: una nota debe estar entre 0 y 10.	Rechaza la fila si la condición es falsa. Insertar una nota de 11 falla; una nota de 9 entra.	La condición se evalúa en cada INSERT y UPDATE, como un examen de entrada para el dato.
`creditos INT DEFAULT 4`	Valores típicos: si no se dice cuántos créditos, la materia vale 4.	Si el INSERT no da valor para la columna, se guarda el DEFAULT (4) en vez de NULL.	Es el valor de fábrica, como el idioma que trae un celular nuevo antes de que lo cambies.
`ALTER TABLE MATERIA` `ADD aula VARCHAR(10)`	Cuando la tabla ya existe y falta una columna nueva.	Agrega la columna aula a MATERIA; las filas que ya estaban quedan con NULL. También existe DROP COLUMN.	ALTER cambia el plano sin tocar los datos guardados. Oracle y SQLServer no llevan la palabra COLUMN.
`DROP TABLE MATRICULA`	Eliminar una tabla que ya no sirve, como una de pruebas.	Borra la tabla completa, estructura y datos, y no hay vuelta atrás. Si otra la referencia, casi todos se niegan.	Quitar la tabla rompería las claves foráneas que apuntan a ella, así que la base protege esos vínculos.

> **EJEMPLO RESUELTO**
> Se crea MATRICULA con PRIMARY KEY (id_est, cod), dos FOREIGN KEY, nota DECIMAL(4,2) y CHECK (nota BETWEEN 0 AND 10). Insertar (1, 'RED', 11) falla por el CHECK; (1, 'BD', 8) falla porque la pareja ya existe; (99, 'BD', 8) falla porque el estudiante 99 no existe. En cambio (5, 'RED', 10) entra. Borrar a Ana (id 1) también falla, porque tiene matrículas.

Tipos comunes: INT (entero), VARCHAR(n) (texto de hasta n caracteres), DECIMAL(p,s) (decimales), DATE (fecha). Oracle usa VARCHAR2 y NUMBER. Cambiar el tipo de una columna se escribe MODIFY en Oracle y MySQL, y ALTER COLUMN en SQLServer y PostgreSQL.

Gráficos de apoyo

MATRICULA

Columna	Tipo	Reglas
id_est	INT	PK + FK a ESTUDIANTE
cod	VARCHAR(8)	PK + FK a MATERIA
nota	DECIMAL(4,2)	CHECK entre 0 y 10

Estructura de MATRICULA
Cada columna con su tipo y las reglas que la protegen.

Datos rechazados

Intento de INSERT	Regla que falla
(1, 'RED', 11)	CHECK
(1, 'BD', 8) repetida	PRIMARY KEY
(99, 'BD', 8)	FOREIGN KEY
estudiante sin nombre	NOT NULL
materia 'Redes' otra vez	UNIQUE

Inserciones que la base rechaza
Cada dato malo choca con una restricción distinta.

Tipos de datos

Tipo	Guarda	Ejemplo	En Oracle
INT	entero	21	NUMBER
VARCHAR(n)	texto	Ana	VARCHAR2(n)
DECIMAL(p,s)	decimales	8.50	NUMBER(p,s)
DATE	fecha	2026-10-16	DATE

Tipos de datos comunes
Lo que guarda cada tipo y cómo se llama en Oracle.

TEMA 17

Manipular datos: INSERT, UPDATE, DELETE y operadores

INSERT agrega filas, UPDATE cambia valores y DELETE borra filas. El WHERE decide a cuáles filas afecta; sin WHERE afecta a todas. Los operadores (comparación, AND, OR, BETWEEN, IN, LIKE, IS NULL) arman esas condiciones.

> **RECUERDA** UPDATE o DELETE sin WHERE es pasar el borrador por todo el pizarrón: primero escribe el WHERE y pruébalo con un SELECT.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`INSERT INTO ESTUDIANTE VALUES (6, 'Eva', 'Redes', 20)`	Cuando llega un estudiante nuevo a la base.	Agrega una fila con valores en el orden de las columnas. Si nombras columnas, las omitidas quedan NULL o DEFAULT.	Antes de guardar, la base revisa todas las restricciones (clave, NOT NULL, CHECK); si una falla, no entra nada.
`UPDATE ESTUDIANTE SET edad = edad + 1 WHERE id_est = 3`	Corregir o actualizar datos, como la edad de un estudiante.	Cambia edad solo en las filas que cumplen el WHERE: Marta pasa de 20 a 21. SET puede llevar varias columnas.	El lado derecho se calcula con el valor viejo de cada fila y el resultado reemplaza al anterior.
`DELETE FROM ESTUDIANTE WHERE id_est = 6`	Dar de baja una fila, como un estudiante que se retira.	Borra las filas que cumplen el WHERE. Sin WHERE vacía la tabla, pero la tabla sigue existiendo (DROP la elimina).	Si otra tabla depende de esa fila por clave foránea, la base no la borra, salvo que se definiera borrado en cascada.
`WHERE edad >= 20 AND carrera <> 'Electronica'`	Combinar varias condiciones en una sola consulta.	Compara con =, <>, <, >, <=, >=. Une con AND (las dos), OR (basta una) y NOT. Salen Luis y Pedro.	AND se evalúa antes que OR; si los mezclas, usa paréntesis para no equivocarte.
`WHERE edad BETWEEN 19 AND 21`	Rangos: edades, fechas o notas entre dos valores.	Verdadero si edad está entre 19 y 21, incluidos los extremos. Salen Ana, Luis y Marta.	Equivale a edad >= 19 AND edad <= 21; es solo una forma más corta de escribirlo.
`WHERE carrera IN ('Electronica', 'Telematica')`	Cuando el valor puede ser uno de varios, como elegir de una lista.	Verdadero si carrera es igual a alguno de la lista. Salen Marta y Sofia.	Es una cadena de OR escrita en corto: carrera = 'Electronica' OR carrera = 'Telematica'.
`WHERE nombre LIKE 'M%'`	Buscar por patrón: nombres que empiezan con M o terminan en a.	El comodín % vale por cualquier texto y _ por una sola letra. '%a' da Ana, Marta y Sofia; '_u%' da Luis.	Compara la forma del texto, no el valor exacto, como buscar contactos que empiezan con M. Las mayúsculas dependen del sistema.
`WHERE nota IS NULL`	Encontrar datos que faltan, como notas aún sin calificar.	Verdadero si nota no tiene valor. Da la matrícula (4, 'PRO'). Lo contrario es IS NOT NULL.	NULL es «no se sabe» y nada es igual a algo desconocido: nota = NULL nunca da verdadero, por eso existe IS NULL.

EJEMPLO RESUELTO

Se agrega a Eva con INSERT (6 estudiantes), se sube un año a Marta con UPDATE (pasa de 20 a 21) y se borra a Eva con DELETE (quedan 5). Luego SELECT nombre FROM ESTUDIANTE WHERE edad BETWEEN 19 AND 21 AND nombre LIKE 'M%' da solo Marta: Ana y Luis cumplen la edad, pero no empiezan con M. Y WHERE nota IS NULL en MATRICULA da la matrícula (4, 'PRO'), aún sin nota.

Los textos van entre comillas simples; las fechas también, en la mayoría de los sistemas. La comparación de mayúsculas en LIKE cambia: MySQL y SQL Server no distinguen por defecto; Oracle y PostgreSQL sí.

Gráficos de apoyo

Antes

id_est	nombre	edad
1	Ana	19
2	Luis	21
3	Marta	20
4	Pedro	22
5	Sofia	18

Después de UPDATE

id_est	nombre	edad
1	Ana	19
2	Luis	21
3	Marta	21
4	Pedro	22
5	Sofia	18

UPDATE antes y después

Solo la fila verde, la de Marta, cambia su edad de 20 a 21.

Filtros sobre ESTUDIANTE

WHERE	Devuelve
edad BETWEEN 19 AND 21	Ana, Luis, Marta
carrera IN ('Electronica','Telematica')	Marta, Sofia
nombre LIKE '%a'	Ana, Marta, Sofia
nombre LIKE '_u%'	Luis

Qué devuelve cada filtro

Cada condición del WHERE y los estudiantes que deja pasar.

nota IS NULL

id_est	cod	nota
4	PRO	NULL

nota = NULL

id_est	cod	nota
(vacio)		

IS NULL frente a = NULL

Comparar con = NULL no encuentra nada; IS NULL sí halla la nota faltante.

TEMA 18

Agregación, agrupamiento, reuniones y operaciones de conjuntos

Las funciones de agregación (COUNT, SUM, AVG, MIN, MAX) resumen muchas filas en un valor. GROUP BY lo hace por grupos y HAVING filtra grupos. JOIN une filas de dos tablas; UNION, INTERSECT y EXCEPT combinan resultados.

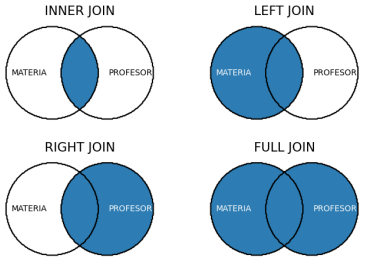
> **RECUERDA** GROUP BY es repartir las hojas de matrícula en montones por materia; la función cuenta o promedia cada montón y HAVING tira los montones que no sirven.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`SELECT COUNT(*), AVG(nota) FROM MATRICULA`	Totales de toda la tabla: cuántas matrículas hay y su nota media.	COUNT(*) cuenta filas (7). SUM, AVG, MIN y MAX ignoran los NULL: AVG(nota) usa solo las 6 notas y da 7.17.	Cada función recorre la columna y devuelve un solo número, así que toda la tabla se vuelve una sola fila.
`GROUP BY cod`	Resúmenes por categoría: promedio por materia, estudiantes por carrera.	Forma un grupo por cada cod y aplica la función a cada grupo: BD 7.0, CAL 7.0 y PRO 8.0.	Primero junta las filas con el mismo cod y luego calcula. Toda columna del SELECT sin función debe estar en el GROUP BY.
`HAVING AVG(nota) > 7`	Quedarte con los grupos que cumplen algo: materias con promedio mayor a 7.	Filtra grupos ya calculados: de BD, CAL y PRO solo PRO (8.0) pasa. WHERE filtra filas; HAVING, grupos.	WHERE actúa antes de agrupar y no puede usar AVG; HAVING actúa después y sí puede.
`FROM MATERIA M INNER JOIN PROFESOR P ON M.id_prof = P.id_prof`	Unir cada materia (M) con su profesor (P): la reunión natural ⋈ de siempre.	Solo salen las parejas que coinciden: 3 filas (BD-Rojas, PRO-Vera, CAL-Mora). Redes y el profesor Paz no aparecen.	Es el producto cruzado filtrado por la condición ON: se descartan todas las parejas que no coinciden.
`FROM MATERIA M LEFT JOIN PROFESOR P ON M.id_prof = P.id_prof`	Cuando no quieres perder filas de la tabla izquierda, como materias sin profesor.	Salen las 3 coincidencias más Redes con NULL en profesor (4 filas). Es la reunión externa izquierda.	Hace el INNER y agrega las filas de la izquierda sin pareja, rellenando con NULL el lado derecho.
`FROM MATERIA M RIGHT JOIN PROFESOR P ON M.id_prof = P.id_prof`	Lo mismo hacia el otro lado: profesores aunque no dicten ninguna materia.	Salen las 3 coincidencias más el profesor Paz con NULL en materia (4 filas). Es el espejo de LEFT JOIN.	Conserva todas las filas de la tabla derecha; es lo mismo que un LEFT JOIN con las tablas intercambiadas.
`FROM MATERIA M FULL JOIN PROFESOR P ON M.id_prof = P.id_prof`	Ver todo a la vez: materias sin profesor y profesores sin materia.	Salen 5 filas: 3 coincidencias, Redes con NULL y Paz con NULL. MySQL no lo tiene: usa LEFT JOIN UNION RIGHT JOIN.	Es la unión de LEFT y RIGHT: no se pierde ninguna fila de ninguno de los dos lados.
`FROM ESTUDIANTE CROSS JOIN MATERIA`	Generar todas las combinaciones posibles, por ejemplo para pruebas de horarios.	Producto cartesiano ×: 5 estudiantes por 4 materias dan 20 filas. No lleva ON.	Empareja cada fila de una tabla con cada fila de la otra; por eso crece muy rápido y casi nunca se usa solo.
`SELECT ... UNION SELECT ...`	Juntar los resultados de dos consultas: estudiantes de BD o de PRO.	UNION junta sin repetir (∪), INTERSECT deja los comunes (∩), EXCEPT resta (−). En Oracle es MINUS.	Las dos consultas deben tener igual número de columnas y tipos parecidos. UNION ALL conserva los repetidos.

> **EJEMPLO RESUELTO**
> Promedio por materia: SELECT cod, AVG(nota) FROM MATRICULA GROUP BY cod. Resultado: BD 7.0 (notas 9, 7 y 5), CAL 7.0 (6 y 8) y PRO 8.0. Pedro tiene NULL en PRO y AVG lo ignora: el promedio de PRO usa solo el 8 de Ana. Con HAVING AVG(nota) > 7 queda solo PRO. Para PRO, COUNT(*) da 2 pero COUNT(nota) da 1.

Con BD = {1, 2, 4} (estudiantes matriculados en BD) y PRO = {1, 4}: UNION da {1, 2, 4}, INTERSECT da {1, 4} y EXCEPT da {2}. MySQL no tiene FULL JOIN y trae INTERSECT y EXCEPT solo desde la versión 8.0.31. Las cuatro reuniones del cuadro usan MATERIA (M) y PROFESOR (P).

Gráficos de apoyo



Diagramas de Venn de los JOIN
En azul, las filas que devuelve cada tipo de reunión entre MATERIA y PROFESOR.

INNER JOIN

materia	profesor
Bases de Datos	Rojas
Programacion	Vera
Calculo	Mora

FULL JOIN

materia	profesor
Bases de Datos	Rojas
Programacion	Vera
Calculo	Mora
Redes	NULL
NULL	Paz

INNER JOIN frente a FULL JOIN
FULL agrega Redes sin profesor y Paz sin materia, con NULL en rojo.

GROUP BY cod

cod	filas	prom
BD	3	7.0
CAL	2	7.0
PRO	2	8.0

HAVING AVG(nota) > 7

cod	filas	prom
PRO	2	8.0

GROUP BY y HAVING
Primero se arma un grupo por materia; HAVING deja solo el promedio mayor a 7.

TEMA 19

Subconsultas y vistas

Una subconsulta es una consulta dentro de otra, entre paréntesis: la de afuera usa su resultado. Una vista es una consulta guardada con nombre que se usa como una tabla y sirve para armar reportes sin repetir la consulta.

RECUERDA La subconsulta es una pregunta dentro de otra pregunta; la vista es dejar esa pregunta pegada en la pared con un nombre, para repetirla sin reescribirla.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`WHERE cod IN (SELECT cod FROM MATRICULA)`	Preguntar si un valor está en una lista que otra consulta calcula: materias con matrículas.	Primero corre la consulta de adentro y luego deja las materias cuyo cod está en esa lista: BD, PRO y CAL.	Es un IN con lista dinámica. Cuidado: NOT IN con un NULL en la lista no devuelve nada; mejor usa NOT EXISTS.
`WHERE nota > (SELECT AVG(nota) FROM MATRICULA)`	Comparar con un valor calculado: notas por encima del promedio general (7.17).	La subconsulta devuelve un solo valor; salen las matrículas con nota mayor: (1,BD,9), (1,PRO,8) y (3,CAL,8).	Se calcula una sola vez y luego se usa como un número fijo, igual que si lo hubieras escrito a mano.
`WHERE NOT EXISTS (SELECT * FROM MATRICULA X WHERE X.cod = M.cod)`	Preguntar si algo existe o no: materias (M) que nadie ha tomado.	EXISTS es verdadero si la subconsulta devuelve al menos una fila; NOT EXISTS, si no devuelve ninguna. Sale Redes.	Solo importa si hay fila, no qué columnas trae. Por eso no sufre el problema del NULL que tiene NOT IN.
`WHERE nota > (SELECT AVG(nota) FROM MATRICULA X WHERE X.cod = M.cod)`	Comparar cada fila con su propio grupo: la nota contra el promedio de su misma materia (M).	La subconsulta usa M.cod de la fila de afuera, así que se recalcula por cada fila. Salen (1,BD,9) y (3,CAL,8).	Es como un bucle: por cada matrícula de afuera, la de adentro saca el promedio de esa materia y compara.
`WHERE nota > ANY (SELECT nota FROM MATRICULA WHERE cod = 'CAL')`	Ser mayor que al menos uno de los valores de una lista.	Las notas de CAL son 6 y 8: basta superar el 6. Salen 4 matrículas: (1,BD), (1,PRO), (2,BD) y (3,CAL).	ANY es un OR entre comparaciones: equivale a ser mayor que el MIN de la lista.
`WHERE nota > ALL (SELECT nota FROM MATRICULA WHERE cod = 'CAL')`	Ser mayor que todos los valores de una lista.	Hay que superar al 6 y al 8, o sea pasar el 8. Solo sale (1,BD,9). Con un NULL en la lista no sale nada.	ALL es un AND entre comparaciones: equivale a ser mayor que el MAX de la lista.
`CREATE VIEW v_notas AS SELECT ...`	Guardar un reporte que se usa seguido: estudiante, materia y nota ya unidos.	Guarda la consulta con el nombre v_notas. No copia datos: cada vez que se usa, corre la consulta de nuevo.	La vista es solo texto, una consulta con nombre. Siempre muestra los datos actuales de las tablas.
`SELECT * FROM v_notas WHERE nota >= 8`	Usar la vista como si fuera una tabla, sin escribir los JOIN otra vez.	Ventajas: simplifica consultas y oculta columnas. Límites: no guarda datos y no siempre admite UPDATE.	Si la vista tiene reuniones o agrupamientos, la base no sabe qué fila real cambiar y rechaza el cambio.

EJEMPLO RESUELTO

Matrículas con nota mayor al promedio de su propia materia: SELECT id_est, cod, nota FROM MATRICULA M WHERE nota > (SELECT AVG(nota) FROM MATRICULA X WHERE X.cod = M.cod). Promedios: BD 7.0, CAL 7.0 y PRO 8.0. Pasan Ana en BD (9 > 7) y Marta en CAL (8 > 7). Ana en PRO tiene 8, que no supera el promedio de PRO (8.0).

La vista del tema une MATRICULA, ESTUDIANTE y MATERIA y deja solo las matrículas que ya tienen nota. DROP VIEW la borra sin tocar las tablas. Oracle y PostgreSQL tienen vistas materializadas, que sí guardan los datos y hay que refrescar. Algunos sistemas, como SQLite, no admiten ANY ni ALL; allí se usa MIN o MAX.

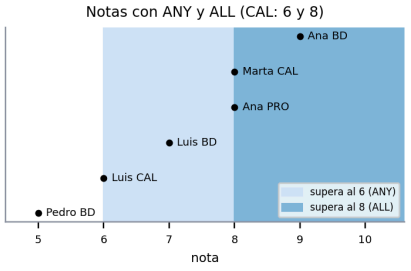
Gráficos de apoyo

nota contra promedio de su materia

id_est	cod	nota	prom
1	BD	9	7.0
1	PRO	8	8.0
2	BD	7	7.0
2	CAL	6	7.0
3	CAL	8	7.0
4	PRO	NULL	8.0
4	BD	5	7.0

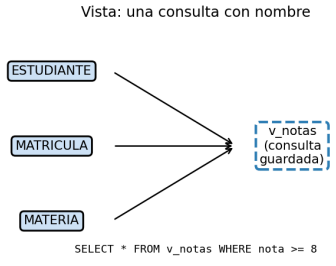
Subconsulta correlacionada

Cada nota se compara con el promedio de su materia; verde si lo supera.



ANY frente a ALL

Con las notas de CAL (6 y 8): ANY pide superar el 6 y ALL pide superar el 8.



La vista guarda la consulta, no los datos

La vista v_{notas} une tres tablas; al consultarla se vuelve a calcular.

PARTE 5

Objetos y estructuras avanzadas

Lo que vive dentro del servidor: procedimientos, disparadores, transacciones e índices.

TEMA 20

Procedimientos almacenados y funciones

Un procedimiento almacenado es un programa con nombre que vive dentro de la base de datos y se llama con CALL. Una función es igual, pero devuelve un valor y se puede usar dentro de un SELECT.

> **RECUERDA** Es una receta guardada en la cocina: en vez de dictar todos los pasos, solo dices el nombre del plato y los ingredientes.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`CREATE PROCEDURE p(IN e INT, OUT r INT)`	Guardar un proceso repetido, como matricular. Varía entre MySQL, Oracle, PostgreSQL y SQL Server.	Crea el procedimiento p: e entra (IN) y r sale (OUT). INOUT hace las dos cosas.	El servidor guarda el programa ya preparado junto a los datos y cualquiera lo llama por su nombre sin reescribirlo.
`BEGIN ... END`	Agrupar varias sentencias que deben ir juntas dentro de un procedimiento o una función.	Lo que va entre `BEGIN` y `END` es el cuerpo. Cada sentencia termina en punto y coma.	Hace el papel de las llaves en otros lenguajes: marca dónde empieza y dónde termina el bloque.
`DECLARE v INT DEFAULT 0`	Guardar un valor temporal, como un promedio o un contador de materias.	Crea la variable v, entera, con valor inicial 0. Se declara al inicio y se cambia con `SET v = v + 1`.	La variable existe solo mientras corre el procedimiento, como una hoja de borrador que se bota al terminar.
`SELECT AVG(nota) INTO v FROM MATRICULA`	Traer un resultado de una consulta a una variable, como el promedio de un estudiante.	Copia en v el promedio de las notas; la consulta debe dar una fila. SQL Server: `SELECT @v = ...`.	`INTO` es el puente entre las tablas y las variables del programa: copia lo que respondió la consulta.
`IF v >= 60 THEN ... ELSE ... END IF`	Decidir según un dato: aprobado o reprobado, con descuento o sin él. Oracle escribe ELSIF.	Si la condición es verdadera corre la primera rama; si no, la de ELSE. ELSEIF agrega más casos.	Es el mismo «si ... entonces ... si no» de cualquier lenguaje, pero ahora corre dentro del servidor.
`WHILE i <= 5 DO ... END WHILE`	Repetir un paso, como revisar las materias de un estudiante. Oracle y PostgreSQL usan LOOP.	Repite el cuerpo mientras la condición sea verdadera. Dentro hay que cambiar i para que el ciclo termine.	La condición se revisa antes de cada vuelta; si nunca se vuelve falsa, el ciclo no acaba nunca.
`CALL aprobar(1, estado)`	Ejecutar el procedimiento desde la consola o la aplicación. Oracle usa EXECUTE; SQL Server, EXEC.	Llama a `aprobar` con el estudiante 1 y recibe en `estado` el valor del parámetro `OUT`.	Solo viajan el nombre y los datos por la red; el trabajo pesado ocurre dentro del servidor, junto a las tablas.
`CREATE FUNCTION f(x INT) RETURNS INT`	Calcular un valor para una consulta, como el costo de una matrícula. MySQL pide DETERMINISTIC.	Crea la función f, que devuelve un entero con RETURN. Se usa: `SELECT f(creditos) FROM MATERIA`.	A diferencia del procedimiento, la función siempre devuelve un valor; por eso cabe en un SELECT o un WHERE.

EJEMPLO RESUELTO

Procedimiento `aprobar(IN e INT, OUT estado VARCHAR(10))`: 1) `SELECT AVG(nota) INTO v` de las notas del estudiante e; 2) si $v \geq 60$, `estado` es «Aprobado», y si no, «Reprobado». Ana tiene 80, 60 y 90: $v = 230/3 \approx 76.7$, entonces «Aprobado». Luis tiene 45 y 55: $v = 50$, entonces «Reprobado». La lógica se comprobó con la consulta equivalente.

Cuándo usarlos: reglas que se repiten, menos viajes por la red y permisos más finos. Cuándo no: la lógica queda atada a un motor (PL/SQL de Oracle, PL/pgSQL de PostgreSQL, T-SQL de SQL Server) y es más difícil de probar. Aquí se usa la sintaxis del estándar SQL/PSM; SQLite no tiene procedimientos.

TEMA 21

Disparadores (triggers)

Un disparador es un programa que la base ejecuta sola cuando ocurre un INSERT, UPDATE o DELETE en una tabla. Nadie lo llama: se activa con el evento. Se define con un momento, un evento y una granularidad.

> **RECUERDA** Es la alarma de una tienda: nadie la prende a mano, suena sola cuando alguien cruza la puerta.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`CREATE TRIGGER aud AFTER UPDATE ON MATRICULA`	Dejar huella de cada cambio de notas. Varía entre motores; el cuerpo cambia en SQL Server y Oracle.	Crea el disparador `aud`: corre AFTER (después) de cada UPDATE en la tabla MATRICULA.	El motor vigila la tabla y, al terminar la operación, ejecuta el cuerpo sin que la aplicación lo pida.

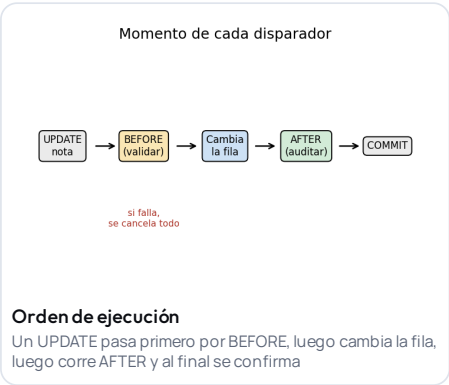
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`BEFORE INSERT ON MATRICULA`	Validar un dato antes de que entre, como una nota fuera de rango.	BEFORE corre antes de grabar la fila y puede rechazarla. AFTER corre cuando la fila ya se grabó.	Es el guardia de la entrada: revisa antes de pasar. El que está en la salida (AFTER) solo anota quién salió.
`UPDATE OF nota ON MATRICULA`	Reaccionar solo si cambia una columna, no cualquier otra de la fila. MySQL no admite OF.	El evento es un UPDATE que toca la columna nota. Los otros eventos son INSERT y DELETE.	Se evita trabajo inútil: cambiar el paralelo de un estudiante no tiene por qué activar la auditoría de notas.
`FOR EACH ROW`	Anotar un cambio por cada fila afectada. SQLite y MySQL solo tienen esta; SQL Server solo la otra.	Granularidad: corre una vez por cada fila. `FOR EACH STATEMENT` corre una sola vez por sentencia.	Si un UPDATE cambia 40 filas, el de fila corre 40 veces y el de sentencia solo 1.
`OLD.nota` y `NEW.nota`	Comparar el valor viejo con el nuevo. Oracle escribe :OLD y :NEW; SQL Server usa deleted e inserted.	OLD es la fila antes del cambio y NEW la fila después. En un INSERT solo hay NEW; en un DELETE, solo OLD.	El motor guarda una copia de la fila antes y otra después, y las deja a la mano dentro del cuerpo.
`WHEN (OLD.nota <> NEW.nota)`	Auditar solo cuando la nota de verdad cambia, no cuando se reescribe el mismo valor.	Condición de disparo: si es falsa, el cuerpo no corre. <> significa «distinto».	Es un filtro previo: así no se llenan las tablas de auditoría con registros que no dicen nada.
`INSERT INTO AUDITORIA VALUES (OLD.nota, NEW.nota)`	Cuerpo típico de auditoría: quién cambió qué y cuándo. Debe ir entre BEGIN y END.	Guarda en `AUDITORIA` el valor anterior y el nuevo, junto con el estudiante y la materia.	La tabla de auditoría solo crece: nadie la edita, así queda un historial que se puede revisar después.
`SIGNAL SQLSTATE '45000'`	Rechazar un registro inválido. PostgreSQL usa RAISE EXCEPTION; Oracle, RAISE_APPLICATION_ERROR.	Lanza un error que cancela la sentencia que disparó el trigger. El 45000 es «error definido por el usuario».	El error sube hasta quien hizo el INSERT y se deshace la sentencia completa; la fila mala nunca entra.

EJEMPLO RESUELTO

Disparador `AFTER UPDATE OF nota ON MATRICULA FOR EACH ROW WHEN (OLD.nota <> NEW.nota)`. Si Ana sube en FISG1002 de 60 a 75, el cuerpo agrega a AUDITORIA la fila (1, FISG1002, 60, 75, UPDATE). Si el UPDATE deja 75 otra vez, el WHEN es falso y no se anota nada. Otro disparador `BEFORE INSERT` rechaza una nota de 110, porque el límite es 100. Verificado en SQLite; en otros motores cambia la forma de escribirlo.

Un disparador es invisible para quien programa la aplicación, y por eso cuesta depurarlo: úsalo para auditoría y reglas simples, no para lógica larga. `INSTEAD OF` *sirve para vistas. Si dos disparadores se activan entre sí, se forma una cadena difícil de seguir.*

Gráficos de apoyo



Orden de ejecución
Un UPDATE pasa primero por BEFORE, luego cambia la fila, luego corre AFTER y al final se confirma



Tabla AUDITORIA
Cada cambio de nota o borrado deja una fila: quién, qué materia, valor anterior, valor nuevo y la acción

TEMA 22

Transacciones, ACID, aislamiento y bloqueos

Una transacción es un grupo de sentencias que se ejecuta como una sola unidad: o se guarda todo o no se guarda nada. Si varias corren a la vez, el nivel de aislamiento decide qué cambios ajenos puedes ver.

RECUERDA Es un retiro y un depósito en el cajero: si el dinero salió de una cuenta pero no llegó a la otra, se cancela toda la operación.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`COMMIT`	Cerrar bien una transferencia entre cuentas. Se abre con START TRANSACTION o BEGIN.	Hace permanentes todos los cambios desde que se abrió la transacción y los muestra a las demás sesiones.	Hasta el COMMIT los cambios son un borrador privado; el COMMIT los vuelve oficiales de un solo golpe.
`ROLLBACK`	Deshacer todo si algo falla a mitad, como un saldo que quedaría negativo.	Descarta los cambios hechos desde el inicio y deja las tablas como estaban antes de la transacción.	La base anota los valores viejos en un registro (log) y los restaura, como el «deshacer» de un editor.

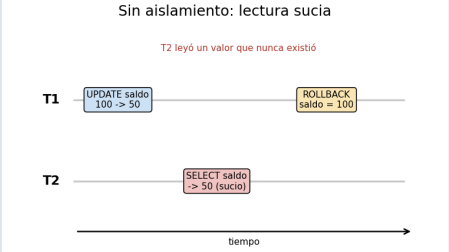
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`SAVEPOINT s1`	Marcar un punto en un proceso largo, como matricular varias materias. SQLServer: SAVE TRANSACTION.	Crea el punto s_1. Luego `ROLLBACK TO SAVEPOINT s1` deshace solo lo posterior y la transacción sigue.	Es un marcador de libro: vuelves a él sin tirar lo anterior. Al final igual hace falta un COMMIT.
$\text{ACID} = \{A, C, I, D\}$	Es la pregunta clásica: qué garantiza una transacción confiable.	A todo o nada, C se cumplen las reglas, I no se estorban entre sí y D lo confirmado permanece.	El registro (log) da A y D, las restricciones dan C y los bloqueos dan I. Cada letra tiene su mecanismo.
`SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`	Elegir entre rapidez y exactitud con muchos usuarios. Los motores no ofrecen los mismos niveles.	Fija qué ves de otros. De menos a más estricto: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE.	Más aislamiento significa más bloqueos y más espera; menos aislamiento va más rápido pero deja pasar errores.
$w_1(x),\ r_2(x),\ \text{ROLLBACK}_1$	Un reporte lee un saldo que otra sesión cambió pero luego canceló.	Lectura sucia: T_2 lee x sin COMMIT de T_1. Si T_1 cancela, T_2 usó un dato que nunca existió.	Es leer un borrador ajeno: si el autor lo rompe, tu decisión quedó basada en nada. READ COMMITTED lo evita.
$r_1(x),\ w_2(x),\ c_2,\ r_1(x)$	Una consulta lee el cupo de una materia dos veces y obtiene números distintos.	No repetible: T_1 lee x, T_2 lo cambia y confirma (c_2), y T_1 lo lee otra vez con otro valor.	Dentro de T_1 el mismo dato cambió entre dos lecturas. REPEATABLE READ lo evita bloqueando las filas leídas.
$r_1(P),\ i_2(y),\ c_2,\ r_1(P)$	Contar los matriculados de un curso dos veces y obtener un total distinto.	Fantasma: T_1 consulta las filas que cumplen P; T_2 inserta una nueva (i_2) y T_1 ve una fila de más.	Las filas que ya leíste están protegidas, pero las que aún no existían no. Solo SERIALIZABLE lo evita del todo.
`SELECT ... FOR UPDATE`	Reservar el último cupo de una materia antes de restarlo. SQL Server usa WITH (UPDLOCK).	Pone un bloqueo exclusivo (X) a las filas leídas: quien quiera escribirlas espera hasta el COMMIT.	El bloqueo compartido (S) deja leer a varios; el exclusivo (X) exige estar solo. S con S convive; S con X y X con X esperan.

EJEMPLO RESUELTO

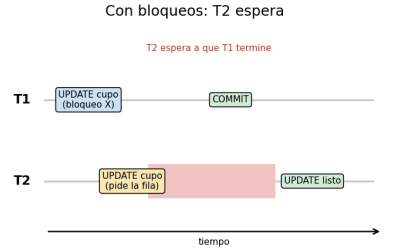
Transferir 100 USD de Ana (500) a Luis (200): `START TRANSACTION`; `UPDATE CUENTA SET saldo = saldo - 100 WHERE id = 1`; el mismo UPDATE con $+100$ para Luis; `COMMIT`. Hasta el COMMIT otra sesión ve 500 y 200; después ve 400 y 300. Si se intenta mover 600, `CHECK (saldo >= 0)` falla en Ana, el ROLLBACK deshace también el depósito a Luis y quedan 500 y 200. Un descuento extra de 50 tras `SAVEPOINT s1` se anula con `ROLLBACK TO s1` y Ana sigue en 400.

Si dos transacciones se esperan una a la otra hay un interbloqueo (deadlock): el motor elige una víctima y le hace ROLLBACK. MySQL (InnoDB) usa REPEATABLE READ por defecto; Oracle, PostgreSQL y SQLServer usan READ COMMITTED. Oracle abre la transacción sola; los otros confirman cada sentencia suelta.

Gráficos de apoyo



Lectura sucia en el tiempo
T2 lee el 50 que T1 escribió sin confirmar; luego T1 cancela y el saldo vuelve a 100



Bloqueo en el tiempo
T1 bloquea la fila con su UPDATE; T2 espera (zona roja) hasta el COMMIT de T1 y recién entonces sigue

Anomalías según el nivel

Nivel	Sucia	No repet.	Fantasma
READ UNCOMMITTED	posible	posible	posible
READ COMMITTED	no	posible	posible
REPEATABLE READ	no	no	posible
SERIALIZABLE	no	no	no

Qué evita cada nivel
Cuanto más abajo en la tabla, menos errores permitidos y más espera. Es lo que fija el estándar SQL

TEMA 23

Índices: árbol B+, hash y costo de una consulta

Un índice es una estructura aparte que guarda los valores de una columna ya ordenados, con la dirección de su fila. Permite hallar una fila sin leer toda la tabla, a cambio de espacio y de trabajo extra al escribir.

RECUERDA Es el índice analítico al final de un libro: buscas la palabra, ves la página y vas directo, en vez de hojear todo.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`CREATE INDEX idx_nombre ON ESTUDIANTE(nombre)`	Acelerar una búsqueda frecuente, como hallar un estudiante por su nombre.	Crea el índice `idx_nombre` sobre la columna `nombre`. Por defecto es un árbol B+.	Los nombres quedan ordenados en un árbol; la búsqueda baja por niveles en vez de leer fila por fila.

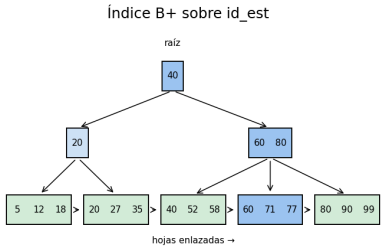
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`CREATE UNIQUE INDEX u_correo ON ESTUDIANTE(correo)`	Impedir repetidos, como dos estudiantes con el mismo correo. PRIMARY KEY y UNIQUE ya lo crean solos.	Es un índice que además rechaza un valor repetido: el segundo **`INSERT`** con ese correo falla.	Al buscar dónde va el valor en el árbol ordenado, el motor ve si ya está. Revisar casi no cuesta nada.
`CREATE INDEX i_cp ON ESTUDIANTE(carrera, promedio)`	Filtros por dos columnas a la vez, como carrera = 'Civil' y promedio mayor a 90.	Ordena por carrera y, dentro de cada una, por promedio. Ayuda si filtras por la primera columna.	Es la guía telefónica ordenada por apellido y luego por nombre: sin el apellido no puedes buscar por nombre.
$h = \lceil \log_m N \rceil$	Estimar cuántas páginas baja una búsqueda en una tabla grande.	h niveles del árbol B+, N filas y m claves por nodo (100 a 300). Con $N = 10^5$ y $m = 300$, $h = 3$.	Cada nivel divide la búsqueda entre m. De mil a mil millones de filas el árbol pasa solo de 2 a 4 niveles.
$b = h(k) \bmod B$	Búsquedas exactas por clave, como un código de materia. PostgreSQL: USING HASH; MySQL: MEMORY.	La función h convierte la clave k en un número; el residuo entre B cubetas da la cubeta de la fila b.	No se recorre nada: se calcula la dirección. Pero claves cercanas caen lejos, así que no sirve para rangos.
$C_{\text{sin}} = \lceil N/f \rceil$	Consulta sin índice en 100 000 filas: el motor lee toda la tabla (SCAN en el plan).	Páginas leídas al recorrer todo: N filas y f filas por página. Con $N = 10^5$ y $f \approx 95$, unas 1050.	Sin orden, la fila puede estar en cualquier página, así que hay que revisarlas todas. Crece igual que N.
$C_{\text{con}} = h + 1$	Búsqueda por igualdad con índice. EXPLAIN muestra SEARCH en SQLite o Index Scan en PostgreSQL.	h páginas del árbol más 1 de la tabla. Con 10^5 filas son unas 4 lecturas contra 1050 sin índice.	Se baja un nivel por página hasta la hoja y se salta a la fila con su dirección; no se mira nada más.
$W = 1 + k$	Tablas con muchas altas, como las matrículas del primer día. Cada índice de más cuesta.	Escrituras por INSERT: la tabla (1) más k índices. Con 3 índices, 100 000 altas tardaron unas 4 veces más.	Cada índice es una copia ordenada: al insertar, el motor debe colocar la clave nueva en su lugar en todas.

EJEMPLO RESUELTO

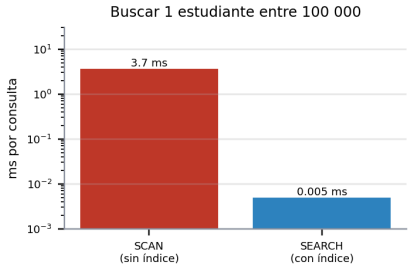
ESTUDIANTE con 100 000 filas ocupa unas 1050 páginas de 4 KB. La consulta **`WHERE nombre = 'Estudiante 77777'`** sin índice da el plan SCAN: revisa las 100 000 filas, unos 3.7 ms. Tras **`CREATE INDEX idx_nombre`** el plan dice SEARCH USING INDEX: baja 3 niveles y lee 1 fila, unos 0.005 ms. Es del orden de mil veces menos tiempo. Con **`LIKE '%7777'`** el índice no ayuda y vuelve el SCAN. Medido en SQLite; el factor varía con el equipo.

En el árbol B+ solo las hojas guardan las direcciones de las filas y están enlazadas, por eso sirve para $=$, $<$, $>$ y BETWEEN. El hash solo sirve para $=$. Un índice no ayuda con `LIKE '%abc'`, con una función sobre la columna ni con tablas muy pequeñas.

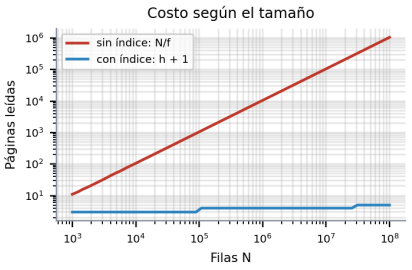
Gráficos de apoyo



Árbol B+ de 5 hojas
La búsqueda del 71 baja por el camino azul: raíz, nodo intermedio y hoja; las hojas están enlazadas



Con y sin índice: tiempo medido
Medido con 100 000 filas: el plan SCAN tardó unos 3.7 ms y el plan SEARCH USING INDEX unos 0.005 ms



Páginas leídas al crecer la tabla
Sin índice las páginas crecen igual que las filas; con índice casi no cambian (escala logarítmica)

PARTE 6

Seguridad y conceptos avanzados

Quién puede hacer qué, y hacia dónde van las bases de datos modernas.

TEMA 24

Seguridad básica e inyección SQL

La seguridad protege los datos de accesos no permitidos, errores y fallas, y se arma por capas: identidad, permisos, cifrado, respaldos y auditoría. La inyección SQL es el ataque más común: alguien escribe SQL dentro de un campo.

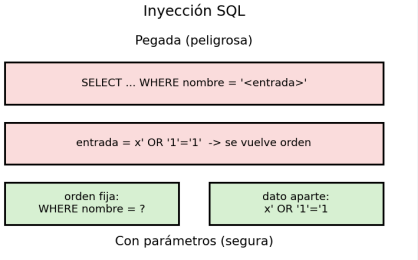
> **RECUERDA** Es un edificio de ESPOL: portería (autenticación), llaves por aula (permisos), caja fuerte (cifrado), copia de llaves (respaldo) y cámaras (auditoría).

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$\text{autenticación} \rightarrow \text{autorización}$	Al entrar al sistema de matrículas: primero demuestras quién eres y luego se revisa qué te toca.	Autenticar es comprobar la identidad (usuario y clave); autorizar es decidir qué puede hacer esa identidad.	Sin saber quién eres no se pueden dar permisos; por eso la identidad va primero y los permisos después.
$h = H(\text{clave} \,\|\| \, \text{sal})$	Guardar las claves de la tabla USUARIO sin guardar la clave real.	H es una función hash (como SHA-256), la sal es un texto al azar de cada usuario y h es lo que se guarda.	El hash no se puede deshacer; la sal hace que dos claves iguales den resultados distintos.
$C = E_k(M), \qquad M = D_k(C)$	Proteger cédulas y notas en el disco y cuando viajan por la red (TLS).	M es el dato legible, k la llave, E cifra, D descifra y C es el dato cifrado que se guarda o se envía.	Sin la llave k, C parece ruido; si roban el disco o espían la red, no sacan nada útil.
3-2-1	Planear las copias de respaldo de la base de la facultad.	Tres copias de los datos, en dos tipos de medio distintos, y una de ellas fuera del edificio.	Un incendio, un disco dañado o un virus rara vez golpean las tres copias a la vez.
`AUDIT SELECT ON estudiante`	Saber quién consultó las notas y cuándo. Es de Oracle; otros motores usan extensiones o bitácoras.	Pide al motor anotar en un registro cada lectura de ESTUDIANTE: usuario, hora y origen.	El registro no evita el ataque, pero deja huella: sin él nadie sabe qué pasó ni quién fue.
`WHERE nombre = 'x' OR '1'='1'`	Inyección en el login: el atacante escribe $x'OR'1'='1$ en el campo nombre.	Como $'1'='1'$ siempre es verdad, la condición deja pasar todas las filas de la tabla.	La consulta se armó pegando texto; el motor no distingue entre lo que escribió el programador y el usuario.
`WHERE nombre = 'admin' --' AND ...`	Inyección con comentario: el atacante escribe $admin'--$ en nombre.	Las dos rayas comienzan un comentario, y la verificación de la clave desaparece de la consulta.	Todo lo que va después de $--$ lo ignora el motor, así que la parte de la clave ya no se evalúa.
`WHERE nombre = ? AND clave = ?`	La defensa principal: toda consulta que reciba datos del usuario (login, buscador, filtros).	Los ? son huecos: el motor recibe la consulta y por separado los valores, que solo se tratan como datos.	Con la estructura ya fijada, un texto raro se busca tal cual como nombre y no puede cambiar la consulta.

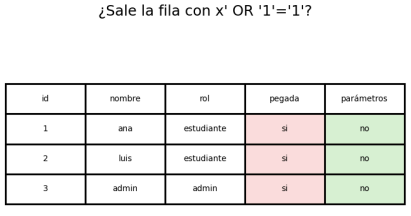
> **EJEMPLO RESUELTO**
> La tabla USUARIO tiene ana, luis y admin. Si la página pega el texto y escribes $x'OR'1'='1'--$ como nombre y cualquier clave, la consulta devuelve las 3 filas y entras sin conocer ninguna clave. Con $admin'--$ devuelves solo la fila de admin. Con parámetros ese mismo texto se busca como nombre y devuelve 0 filas. Con ana y su clave real devuelve 1 fila.

El signo del parámetro cambia según el motor: ? en MySQL y SQLite, $:nombre$ en Oracle y variables numeradas en PostgreSQL. Además: valida la entrada y da a la aplicación un usuario con pocos permisos.

Gráficos de apoyo



Consulta pegada y con parámetros
Con texto pegado, lo escrito se mezcla con la orden; con parámetros viaja aparte como dato



Qué devuelve el ataque
La tabla USUARIO y las filas que salen con el texto malicioso, pegado o con parámetros

TEMA 25

Administración de usuarios y privilegios

Un usuario es una cuenta y un privilegio es un permiso concreto (SELECT, INSERT, UPDATE, DELETE) sobre un objeto. GRANT da permisos, REVOKE los quita y un rol agrupa permisos para darlos a muchos usuarios de una vez.

> **RECUERDA** Es el llavero de la facultad: a la secretaria no le das la llave maestra sino solo las de su oficina, y si cambia de cargo le cambias el llavero (el rol).

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`CREATE USER ana IDENTIFIED BY 'x7k2'`	Dar de alta una cuenta. Esa forma sirve en Oracle y MySQL; PostgreSQL usa *WITH PASSWORD*.	Crea al usuario *ana* con su clave. Recién creada, no puede hacer casi nada, ni leer tus tablas.	El SQL estándar casi no define usuarios; cada motor lo hace a su manera, pero siempre parte sin permisos.
`GRANT SELECT ON estudiante TO ana`	Que un docente pueda consultar la lista de estudiantes sin poder cambiarla.	Otorga a *ana* el permiso de leer la tabla ESTUDIANTE. Se pueden dar varios: *SELECT*, *INSERT*.	El motor guarda cada permiso como una tripleta: quién, qué acción y sobre qué objeto; sin ella, rechaza.
`GRANT UPDATE (promedio) ON estudiante TO ana`	Permitir editar una sola columna, por ejemplo solo el promedio y no el nombre.	El permiso se limita a la columna escrita entre paréntesis; las demás columnas quedan protegidas.	Mientras más fino el permiso, menos daño puede hacer un error o una cuenta robada.
`GRANT SELECT ON matricula TO ana WITH GRANT OPTION`	Delegar: ana podrá pasar este mismo permiso a otros (en Oracle, para roles se usa *ADMIN OPTION*).	Da el permiso y además el derecho de repartirlo a otros usuarios.	La confianza se hereda: si quitas el permiso a ana, también se retira lo que ella repartió.
`REVOKE INSERT ON matricula FROM ana`	Quitar un permiso cuando alguien cambia de funciones o termina su contrato.	Retira a *ana* el permiso de insertar en MATRICULA. Los demás permisos que tenga se quedan igual.	Es el inverso exacto de GRANT: borra esa tripleta de la lista de permisos del motor.
`CREATE ROLE secretaria`	Crear un grupo de permisos para un cargo que ocupan varias personas.	Crea el rol *secretaria*, vacío. Luego se le dan permisos con GRANT, igual que a un usuario.	Un rol es un usuario \"sin cuenta\": solo sirve para agrupar permisos y repartirlos juntos.
`GRANT secretaria TO ana`	Asignar el cargo a una persona: ana recibe de golpe todos los permisos del rol.	Da a *ana* el rol *secretaria*. Si luego al rol se le quita un permiso, ana también lo pierde.	Los permisos viven en el rol, no en cada persona: cambiar el rol cambia a todos sus miembros a la vez.
$P_{\text{dados}} \subseteq P_{\text{necesarios}}$	Criterio al decidir permisos: nunca más de lo que el trabajo de esa persona exige.	Principio de mínimo privilegio: el conjunto de permisos dados cabe dentro del que la tarea necesita.	Si la cuenta se roba o se equivoca, el daño llega solo hasta donde llegaban sus permisos.

> **EJEMPLO RESUELTO**
> Creas el rol *secretaria* con SELECT e INSERT sobre MATRICULA y SELECT sobre ESTUDIANTE: 3 privilegios. Se lo das a ana. Ana consulta ESTUDIANTE e inserta matrículas, pero DELETE sobre MATRICULA se rechaza. Luis, sin rol, ni consulta. Luego `REVOKE INSERT ON matricula FROM secretaria`: el rol queda con 2 privilegios y ana ya no inserta, aunque sigue consultando. Un solo REVOKE cambió a todas las secretarias.

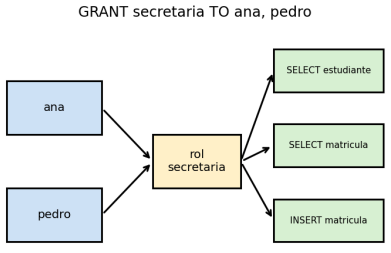
SQLite no tiene GRANT; las sentencias siguen el estándar y se usan en MySQL, Oracle y PostgreSQL. En SQL Server se crea primero un LOGIN. Para borrar una cuenta: `DROP USER ana`.

Gráficos de apoyo

Privilegios por cuenta

cuenta	ESTUDIANTE	MATRICULA
rol secretaria	S	S I
ana (por el rol)	S	S I
luis (sin rol)	-	-
administrador	S I U D	S I U D

Matriz de privilegios
Qué puede hacer cada cuenta sobre cada tabla (S = SELECT, I = INSERT, U = UPDATE, D = DELETE)



Los roles reparten permisos
Los permisos se dan al rol y los usuarios los reciben al entrar al rol

TEMA 26

Bases de datos NoSQL: tipos, teorema CAP y BASE

NoSQL agrupa las bases que no usan tablas de esquema fijo: clave-valor, documentos, columnas y grafos. Sirven para repartir datos enormes entre muchos servidores, y a cambio relajan la consistencia estricta de ACID.

> **RECUERDA** Cuatro tipos de archivador: un casillero con llave, carpetas de hojas sueltas, hojas por columna y un mapa de hilos que unen a las personas.

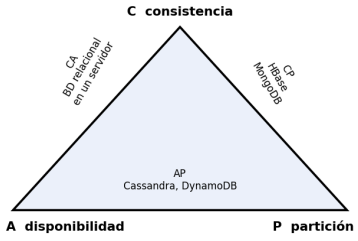
REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
$k \mapsto v$	Sesiones de login, carritos de compra y cachés (Redis, DynamoDB).	Base clave-valor: la clave única k apunta a un valor v que el sistema guarda sin mirar adentro.	Buscar por clave es abrir un casillero numerado: vas directo, sin recorrer nada, por eso es muy rápido.
`{nombre: 'Ana', materias: ['BD', 'Calc']}`	Perfiles y catálogos donde cada elemento tiene campos distintos (MongoDB, CouchDB).	Base de documentos: un objeto completo, con listas incluidas, se guarda en un solo documento tipo JSON.	Lo que se lee junto se guarda junto y se evitan los JOIN; el costo es repetir datos en varios documentos.
$(\text{fila},\ \text{familia:col},\ t) \mapsto v$	Registros enormes con millones de eventos: sensores, bitácoras, mensajes (Cassandra, HBase).	Base de columnas: cada valor v se ubica por su fila, su columna y la marca de tiempo t en que se escribió.	Guardar por columnas permite leer solo lo que se pide y repartir las filas entre servidores.
$G = (V,\ E)$	Redes sociales, rutas y recomendaciones: amigos de amigos (Neo4j).	Base de grafos: V son los nodos (personas) y E las aristas (relaciones, como «sigue a»).	La relación se guarda como dato y no se calcula con JOIN; recorrer vecinos es seguir enlaces ya hechos.
$\{C,\ A,\ P\} \rightarrow \text{solo 2 a la vez}$	Al diseñar una base repartida en varios servidores (teorema CAP, de Brewer).	C: todos ven el mismo dato; A: siempre se responde; P: sigue funcionando aunque la red se parta.	En una red real los cortes ocurren, así que P no se puede evitar; queda elegir entre C y A.
$P \Rightarrow C \text{ o } A$	Cuando un cable o servidor falla y dos nodos no pueden hablarse.	Con partición debes elegir: consistencia (esperar o rechazar) o disponibilidad (responder con lo que hay).	Si no se pueden comunicar, o esperas a ponerte de acuerdo (pierdes A) o respondes ya (pierdes C).
BASE frente a ACID	Elegir el comportamiento: un banco pide ACID; un «me gusta» de una red social acepta BASE.	BASE: Básicamente disponible, Estado suave, consistencia Eventual; las copias se igualan con el tiempo.	Aceptas ver un dato viejo por instantes a cambio de que el sistema nunca se caiga ni se bloquee.
$R + W > N$	Bases con N copias (Cassandra, Dynamo) cuando quieres leer el dato más reciente.	N copias, W confirman cada escritura y R se consultan al leer. Si $R + W > N$ se ve la última escritura.	Un grupo de W copias y uno de R siempre comparten alguna copia, y esa tiene el dato nuevo.

EJEMPLO RESUELTO

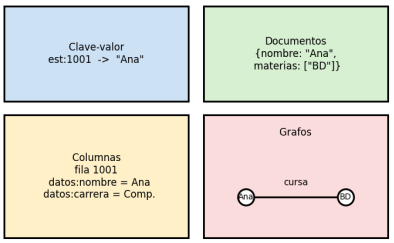
Hay $N = 3$ copias del cupo de una materia. Con $W = 2$ y $R = 2$: $R + W = 4 > 3$, así que cualquier lectura toca al menos una copia con el último valor (comprobado con todas las combinaciones posibles). Con $W = 1$ y $R = 1$: $R + W = 2 \leq 3$, y una lectura puede caer en una copia que aún no se actualizó: es consistencia eventual, más rápida pero con riesgo de leer un cupo viejo.

CAP no dice «elige dos para siempre»: dice que, durante una partición, no se puede tener C y A a la vez. Sistemas CP: HBase, MongoDB (por defecto). Sistemas AP: Cassandra, DynamoDB.

Gráficos de apoyo



Triángulo CAP
Con partición de red hay que elegir; cada tipo de sistema se queda con dos de las tres letras



Tipos de NoSQL con datos
El mismo estudiante guardado en los cuatro tipos de base



ACID frente a BASE
Comparación de las dos filosofías de consistencia

TEMA 27

Data Warehouse: OLAP, esquema en estrella y ETL

Un data warehouse es una base grande, separada de la operativa, hecha para analizar la historia. Los datos entran por un proceso ETL y se organizan en una tabla de hechos (números) rodeada de dimensiones (quién, cuándo, qué).

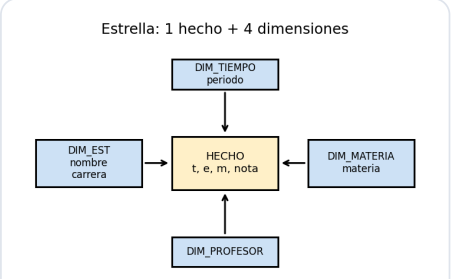
> **RECUERDA** La secretaría (OLTP) anota cada matrícula al instante; el archivo de la facultad (OLAP) junta años de datos para responder cuántos aprobaron por carrera.

REGLA O SENTENCIA	DÓNDE SE USA	PARA QUÉ SIRVE	POR QUÉ FUNCIONA
`UPDATE matricula SET nota = 8.5 WHERE id = 7`	OLTP: el trabajo diario, muchas operaciones cortas sobre pocas filas (matricular, pagar).	Procesamiento transaccional: cada sentencia cambia un registro y debe terminar rápido.	La base operativa se diseña para escribir sin errores ni repetidos (normalizada), no para leer años de historia.
$\text{HECHO}(\underline{t^*, e^*, m^*}, \text{nota})$	La tabla central de la estrella: una fila por cada nota de un estudiante en una materia y periodo.	La clave junta las foráneas t, e, m de las dimensiones; nota es la medida (un número).	Cada fila es un hecho medible; las claves dicen en qué contexto ocurrió y la medida se suma o promedia.
$\text{DIM_EST}(\underline{e}, \text{nombre}, \text{carrera})$	Una de las tablas que rodean al hecho, como tiempo, estudiante o materia.	Dimensión: describe el contexto con muchos atributos de texto. Se usa para filtrar y agrupar los hechos.	Se guarda sin normalizar, con la carrera escrita en cada estudiante, para evitar JOIN y consultar rápido.
`FROM hecho h JOIN dim_est e ON h.e = e.e`	Consulta OLAP típica: un JOIN del hecho con cada dimensión y luego un GROUP BY.	Une cada nota con los datos de su estudiante para poder agrupar por carrera, y así con las demás dimensiones.	En la estrella todo JOIN es de un solo salto, hecho a dimensión; por eso las consultas son simples y rápidas.
$\text{DIM_MATERIA} \xrightarrow{\text{FK}} \text{DIM_FACULTAD}$	Copo de nieve: la dimensión se parte en tablas normalizadas, como materia que apunta a facultad.	Ahorra espacio y evita repetir la facultad, pero cada consulta necesita un JOIN más.	Es una estrella normalizada: gana orden y pierde velocidad. Se usa cuando la dimensión es muy grande.
$E \to T \to L$	Cargar el almacén cada noche desde matrículas, pagos y otros sistemas.	Extraer los datos de las fuentes, Transformarlos (limpiar, unificar formatos) y Cargarlos al almacén.	Cada fuente guarda los datos a su modo; el paso T las vuelve iguales antes de entrar al almacén.
2^n	Cubo de datos: contar formas de agrupar con n dimensiones (GROUP BY CUBE en Oracle y SQL Server).	Con n dimensiones hay 2^n agrupaciones, desde el total general hasta el detalle completo.	Cada dimensión puede estar o no en el GROUP BY: dos opciones por dimensión que se multiplican.
$\gamma_{\text{carrera};\, \text{AVG}(\text{nota})}$	Roll-up: subir de nivel, del detalle por estudiante al promedio por carrera.	Agrupa por carrera y promedia la nota. Bajar de nivel otra vez se llama drill-down.	Subir de nivel es agrupar con menos columnas; bajar es agrupar con más. Filtrar un valor es hacer un slice.

> **EJEMPLO RESUELTO**
> Estrella con HECHO de 6 filas (2 periodos, 3 estudiantes, 2 materias). La consulta junta con DIM_EST y agrupa por carrera: Computacion promedia 7.5 y Electronica 9.0. Un slice con periodo 2025-1 da 2 notas de Computacion y 1 de Electronica. Con 3 dimensiones hay $2^3 = 8$ agrupaciones del cubo. El hecho podría tener a lo sumo $2 \cdot 3 \cdot 2 = 12$ filas y tiene 6.

OLTP y OLAP no se mezclan: el almacén se carga en horas de poco uso para no frenar la base operativa. Un data mart es un almacén pequeño, de una sola área.

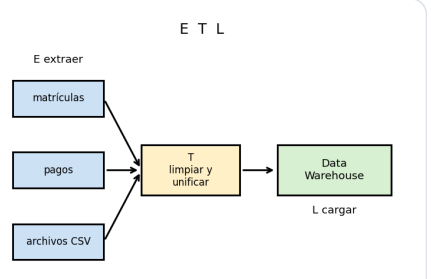
Gráficos de apoyo



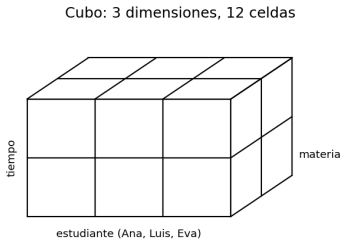
Esquema en estrella
El hecho en el centro y una dimensión por cada punta; cada flecha es una clave foránea



OLTP frente a OLAP
La base operativa y el almacén se diseñan para trabajos distintos



Proceso ETL
Los datos viajan de las fuentes al almacén y de ahí a los reportes



Cubo de datos

Tres dimensiones forman un cubo; cada celda es una medida (2 periodos x 3 estudiantes x 2 materias)

PARTE 7

Formulario general

Lo más importante de la guía en una sola consulta, por parte y por tema.

PARTE 1 Modelo conceptual de datos

1 Introducción a los sistemas de bases de datos

datos + contexto = información	Dato e información
ESTUDIANTE(*matricula*, *nombre*, *carrera*)	Esquema
{(1001, Ana, Computación), ...}	Instancia
externo ↔ conceptual ↔ interno	Tres niveles
`CREATE VIEW v1 AS SELECT nombre FROM ESTUDIANTE`	Nivel externo:
`ALTER TABLE ESTUDIANTE ADD correo VARCHAR(60)`	Independencia lóg
`CREATE INDEX idx_nombre ON ESTUDIANTE(nombre)`	Independencia fís

2 Entidades y atributos

entidad → tabla, atributo → columna	Entidad y atributo
$nombre = (nombres,\ apellidos)$	Atributo compuesto
$telefono = \{t_1,\ t_2,\ \ldots\}$	Atributo multivaluado
$edad = \lfloor (hoy - fnac)/365.25 \rfloor$	Atributo derivado
$\forall e_1 \neq e_2 :\ e_1[K] \neq e_2[K]$	Clave sin repetidos
$e[K] \neq \text{NULL}$	Clave nunca nula
AULA($\underline{cod_edif^*,\ num_aula}$, *capacidad*)	Entidad débil
`ON DELETE CASCADE`	Dependencia de existencia

3 Relaciones entre entidades

$\text{grado}(R) = n$	Grado
$\#\{\, b : (a,b) \in R \}$	Cardinalidad
$A\ (1{:}1)\ B$	Relación 1:1
$A\ (1{:}N)\ B$	Relación 1:N
$A\ (M{:}N)\ B$	Relación M:N
total: $\forall a\ \exists b\ (a,b) \in R$	Participación
MATERIA $\xrightarrow{\text{requiere}}$ MATERIA	Relación recursiva
$R(E_1,\ E_2,\ E_3)$	Relación ternaria

4 Restricciones y notaciones del diagrama

$(0,N) \quad (1,1) \quad (1,N)$	Cardinalidad (mín, máx)
`CHECK (nota BETWEEN 0 AND 100)`	Restricción de dominio
`correo VARCHAR(60) UNIQUE`	Unicidad
`nombre VARCHAR(40) NOT NULL`	Obligatoriedad
$\square - \lozenge - \square$	Notación de Chen
$\mid = 1, \quad \circ = 0, \quad \prec = N$	Notación pata de gallo
PERSONA ⊃ {ESTUDIANTE, PROFESOR}	Supertipo y subtipos
(d) disjunta $\quad (o)$ solapada	Disjunta o solapada

PARTE 2 Bases de datos relacionales

5 Tablas, filas y columnas

$R(A_1, A_2, \ldots, A_n)$	Esquema de relación
$t = \langle v_1, \ldots, v_n \rangle,\ v_i \in \text{dom}(A_i)$	Tupla y dominio
$r \subseteq D_1 \times D_2 \times \cdots \times D_n$	Relación como conjunto
$\text{grado}(R) = n$	Grado
$\text{card}(r) = \lvert r \rvert$	Cardinalidad
`promedio IS NULL`	Detectar NULL
`NULL = NULL`	Lógica de tres valores
`COUNT(promedio)`	Agregación ignora NULL

6 Claves primarias y foráneas

$t_1 \neq t_2 \Rightarrow t_1[SK] \neq t_2[SK]$	Superclave
$\nexists K' \subsetneq K :\ K'$ es superclave	Clave candidata
`PRIMARY KEY` = `UNIQUE` + `NOT NULL`	Clave primaria
$\pi_{FK}(R_1) \subseteq \pi_{PK}(R_2)$	Integridad referencial
`FOREIGN KEY (id_prof) REFERENCES PROFESOR`	FOREIGN KEY
`ON DELETE CASCADE`	ON DELETE CASCADE
`ON DELETE SET NULL`	ON DELETE SET NULL
`ON UPDATE CASCADE`	ON UPDATE CASCADE

7 Dependencias funcionales y cierre

$X \to Y$	Dependencia funcional
$Y \subseteq X \Rightarrow X \to Y$	Reflexividad
$X \to Y \Rightarrow XZ \to YZ$	Aumento
$X \to Y,\ Y \to Z \Rightarrow X \to Z$	Transitividad
$X \to Y,\ X \to Z \Rightarrow X \to YZ$	Unión y descomposición
$X^+ \leftarrow X^+ \cup Z \quad (Y \subseteq X^+,\ Y \to Z)$	Cierre de atributos
$X^+ = R \iff X$ superclave	Prueba de superclave
$X^+ = R,\ \ (X - A)^+ \neq R\ \ \forall A \in X$	Clave por cierre

8 Formas normales y descomposición

$\forall A \in R :\ t[A]$ es un solo valor	1FN
$X \subsetneq K \Rightarrow X \nrightarrow A$	2FN
$X \to A \Rightarrow X$ superclave $\vee$ A primo	3FN
$X \to A \Rightarrow X$ es superclave	FNBC
$R_1 \cap R_2 \to R_1 \ \vee\ R_1 \cap R_2 \to R_2$	Descomposición sin pérdida
$R = R_1 \bowtie R_2 \bowtie \cdots \bowtie R_k$	Reconstruir con reunión
$(F_1 \cup F_2 \cup \cdots)^+ = F^+$	Preservación de dependencias
`UPDATE GENERAL SET depto = 'X' WHERE prof = 'Paz'`	Anomalía de act

9 Del diagrama E-R al esquema relacional

ESTUDIANTE($\underline{id_est}, nombre, carrera$)	Entidad fuerte
MATERIA($\underline{cod_mat}, nombre, id_prof^*$)	Relación 1:N
MATRICULA($\underline{id_est^*, cod_mat^*, periodo}, nota$)	Relación M:N
OFICINA($\underline{cod_ofi}, id_prof^*$ UNIQUE)	Relación 1:1
PARALELO($\underline{cod_mat^*, numero}, cupo$)	Entidad débil
TELEFONO($\underline{id_est^*, telefono}$)	Atributo multivaluado
BECADO($\underline{id_est^*}, porcentaje$)	Generalización: 1 por clase
ESTUDIANTE($\underline{id_est}, tipo, porcentaje, \ldots$)	Generalización: tabla única

PARTE 3 Álgebra relacional

10 Operaciones básicas: selección, proyección, renombrar y producto

$\sigma_{carrera=\text{Computación}}(E)$	Selección
$\sigma_{prom\geq 8 \,\wedge\, carrera=\text{Electrónica}}(E)$	Selección compuesta
$\pi_{nombre,\, carrera}(E)$	Proyección
$\pi_{nombre}(\sigma_{prom>8}(E))$	Composición
$\rho_{X}(E)$	Renombrar tabla
$\rho_{S(codigo,\, alumno)}(R)$	Renombrar columnas
$E \times M$	Producto cartesiano

11 Operadores de conjuntos: unión, intersección y diferencia

$R \cup S$	Unión
$R \cap S$	Intersección
$R - S$	Diferencia
$R - S \neq S - R$	Orden en la diferencia
$R \cap S = R - (R - S)$	Intersección derivada
$\lvert R \cup S \rvert = \lvert R \rvert + \lvert S \rvert - \lvert R \cap S \rvert$	Conteo de la unión
$\text{dom}(A_i) = \text{dom}(B_i) \quad \forall i$	Compatibilidad de unión

12 Operaciones derivadas: reuniones (joins) y semirreunión

$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$	Reunión theta
$E \bowtie_{E.id_est\, =\, N.id_est} N$	Equirreunión
$E \bowtie N$	Reunión natural
$R \overset{\text{izq}}{\bowtie} S$	Reunión externa izquierda
$R \overset{\text{der}}{\bowtie} S$	Reunión externa derecha
$R \overset{\text{comp}}{\bowtie} S$	Reunión externa completa
$R \ltimes S = \pi_{\text{atr}(R)}(R \bowtie S)$	Semirreunión
$\pi_{nombre,\, materia}(E \bowtie N \bowtie M)$	Reunión encadenada

13 División y consultas «para todo»

$R(A, B) \div S(B)$	División
$\pi_A(R) - \pi_A\big((\pi_A(R) \times S) - R\big)$	División con básicas
$\pi_{id_est,cod}(N) \div \pi_{cod}(\sigma_{cred=4}(M))$	División: ejemplo
$\text{atr}(R \div S) = \text{atr}(R) - \text{atr}(S)$	Columnas del cociente
$(R \div S) \times S \subseteq R$	Prueba de la división
$\pi_{id_est}(R \bowtie S)$	Alguna frente a todas
$\pi_{id_est}(E) - (R \div S)$	Los que no cumplen
`NOT EXISTS (... NOT EXISTS (...))`	División en SQL

14 Agregación, agrupamiento y optimización de consultas

$\gamma_{\text{COUNT}(*),\, \text{MAX}(prom)}(E)$	Agregación sin grupos
$_{carrera}\gamma_{\text{COUNT}(*),\, \text{AVG}(prom)}(E)$	Agrupamiento
$_{id_est}\gamma_{\text{COUNT}(cod),\, \text{AVG}(nota)}(N)$	Agrupar por estudiante
$\sigma_{n\geq 2}\big({}_{id_est}\gamma_{\text{COUNT}(cod)\to n}(N)\big)$	Filtrar grupos
$\text{AVG}(x) = \dfrac{\text{SUM}(x)}{\text{COUNT}(x)}$	Promedio y NULL
$\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$	Cascada de selecciones
$\sigma_{c}(R \bowtie S) = \sigma_{c}(R) \bowtie S$	Empujar la selección
$\sigma_{c}(R \times S) = R \bowtie_{c} S$	Producto más selección

PARTE 4

Lenguaje estructurado de consultas (SQL)

15 De álgebra relacional a SQL: SELECT, renombrar, DISTINCT y ORDER BY

$\pi_{\text{nombre}}(\sigma_{\text{edad}>20}(\text{ESTUDIANTE}))$	De álgebra a SQL
`SELECT nombre, carrera`	SELECT: proyección
`WHERE edad > 20`	WHERE: selección
`SELECT nombre AS alumno`	Renombrar columna (AS)
`SELECT DISTINCT carrera`	DISTINCT: sin repetidos
`ORDER BY edad DESC, nombre`	ORDER BY: ordenar

16 Objetos simples: CREATE, ALTER, DROP, tipos y restricciones

`CREATE TABLE MATRICULA (...)`	CREATE TABLE
`PRIMARY KEY (id_est, cod)`	PRIMARY KEY
`FOREIGN KEY (cod)` `REFERENCES MATERIA(cod)`	FOREIGN KEY
`nombre VARCHAR(40) NOT NULL`	NOT NULL
`CHECK (nota BETWEEN 0 AND 10)`	CHECK
`creditos INT DEFAULT 4`	DEFAULT
`ALTER TABLE MATERIA` `ADD aula VARCHAR(10)`	ALTER TABLE
`DROP TABLE MATRICULA`	DROP TABLE

17 Manipular datos: INSERT, UPDATE, DELETE y operadores

`INSERT INTO ESTUDIANTE` `VALUES (6, 'Eva', 'Redes', 20)`	INSERT

`UPDATE ESTUDIANTE`
`SET edad = edad + 1`
`WHERE id_est = 3`
	UPDATE

`DELETE FROM ESTUDIANTE` `WHERE id_est = 6`	DELETE
`WHERE edad >= 20 AND` `carrera <> 'Electronica'`	Comparar y combinar
`WHERE edad BETWEEN 19 AND 21`	BETWEEN
`WHERE carrera IN` `('Electronica', 'Telematica')`	IN
`WHERE nombre LIKE 'M%'`	LIKE
`WHERE nota IS NULL`	IS NULL

18 Agregación, agrupamiento, reuniones y operaciones de conjuntos

`SELECT COUNT(*), AVG(nota)` `FROM MATRICULA`	Funciones de agregación
`GROUP BY cod`	GROUP BY
`HAVING AVG(nota) > 7`	HAVING

`FROM MATERIA M`
`INNER JOIN PROFESOR P`
`ON M.id_prof = P.id_prof`
	INNER JOIN

`FROM MATERIA M`
`LEFT JOIN PROFESOR P`
`ON M.id_prof = P.id_prof`
	LEFT JOIN

`FROM MATERIA M`
`RIGHT JOIN PROFESOR P`
`ON M.id_prof = P.id_prof`
	RIGHT JOIN

`FROM MATERIA M`
`FULL JOIN PROFESOR P`
`ON M.id_prof = P.id_prof`
	FULL JOIN

`SELECT ... UNION` `SELECT ...`	UNION, INTERSECT, EXCEPT

19 Subconsultas y vistas

`WHERE cod IN` `(SELECT cod FROM MATRICULA)`	IN con subconsulta
`WHERE nota >` `(SELECT AVG(nota) FROM MATRICULA)`	Subconsulta escalar

`WHERE NOT EXISTS`
`(SELECT * FROM MATRICULA X`
`WHERE X.cod = M.cod)`
	EXISTS y NOT EXISTS

`WHERE nota >`
`(SELECT AVG(nota)`
`FROM MATRICULA X`
`WHERE X.cod = M.cod)`
	Subconsulta correlacionada

`WHERE nota > ANY`
`(SELECT nota FROM MATRICULA`
`WHERE cod = 'CAL')`
	ANY

`WHERE nota > ALL`
`(SELECT nota FROM MATRICULA`
`WHERE cod = 'CAL')`
	ALL

`CREATE VIEW v_notas AS` `SELECT ...`	CREATE VIEW
`SELECT * FROM v_notas` `WHERE nota >= 8`	Usar una vista

PARTE 5 Objetos y estructuras avanzadas

20 Procedimientos almacenados y funciones

`CREATE PROCEDURE p(IN e INT, OUT r INT)`	Crear procedimiento
`DECLARE v INT DEFAULT 0`	Variable local
`SELECT AVG(nota) INTO v FROM MATRICULA`	SELECT INTO
`IF v >= 60 THEN ... ELSE ... END IF`	Condicional IF
`WHILE i <= 5 DO ... END WHILE`	Ciclo WHILE
`CALL aprobar(1, estado)`	Llamar procedimiento
`CREATE FUNCTION f(x INT) RETURNS INT`	Crear función

21 Disparadores (triggers)

`CREATE TRIGGER aud AFTER UPDATE ON MATRICULA`	Crear dispara
`BEFORE INSERT ON MATRICULA`	BEFORE y AFTER
`UPDATE OF nota ON MATRICULA`	Evento y columna
`FOR EACH ROW`	Granularidad
`OLD.nota y NEW.nota`	OLD y NEW
`WHEN (OLD.nota <> NEW.nota)`	Condición WHEN
`INSERT INTO AUDITORIA VALUES (OLD.nota, NEW.nota)`	
`SIGNAL SQLSTATE '45000'`	Rechazar con error

22 Transacciones, ACID, aislamiento y bloqueos

`COMMIT`	COMMIT
`ROLLBACK`	ROLLBACK
`SAVEPOINT s1`	SAVEPOINT
$\text{ACID} = \{A, C, I, D\}$	Propiedades ACID
`SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`	Nivel de aislamient
$w_1(x),\ r_2(x),\ \text{ROLLBACK}_1$	Lectura sucia
$r_1(x),\ w_2(x),\ c_2,\ r_1(x)$	Lectura no repetible
$r_1(P),\ i_2(y),\ c_2,\ r_1(P)$	Fantasma
`SELECT ... FOR UPDATE`	Bloqueo FOR UPDATE

23 Índices: árbol B+, hash y costo de una consulta

`CREATE INDEX idx_nombre ON ESTUDIANTE(nombre)`	Crear ín
`CREATE UNIQUE INDEX u_correo ON ESTUDIANTE(correo)`	
`CREATE INDEX i_cp ON ESTUDIANTE(carrera, promedio)`	Índice
$h = \lceil \log_m N \rceil$	Altura del árbol B+
$b = h(k) \bmod B$	Índice hash
$C_{\text{sin}} = \lceil N/f \rceil$	Costo sin índice
$C_{\text{con}} = h + 1$	Costo con índice
$W = 1 + k$	Costo de escribir

PARTE 6 Seguridad y conceptos avanzados

24 Seguridad básica e inyección SQL

$\text{autenticación} \rightarrow \text{autorización}$	Autenticar y autorizar
$h = H(\text{clave} \,\|\, \text{sal})$	Hash con sal
$C = E_k(M), \qquad M = D_k(C)$	Cifrado con llave
3-2-1	Regla de respaldo 3-2-1
`AUDIT SELECT ON estudiante`	Auditoría
`WHERE nombre = 'x' OR '1'='1'`	Inyección con OR
`WHERE nombre = 'admin' --' AND ...`	Inyección con comentario
`WHERE nombre = ? AND clave = ?`	Consulta con parámetros

25 Administración de usuarios y privilegios

`CREATE USER ana IDENTIFIED BY 'x7k2'`	Crear usuario
`GRANT SELECT ON estudiante TO ana`	GRANT sobre una tabla
`GRANT UPDATE (promedio) ON estudiante TO ana`	GRANT sobre una colun
`GRANT SELECT ON matricula TO ana WITH GRANT OPTION`	Con opció
`REVOKE INSERT ON matricula FROM ana`	REVOKE
`CREATE ROLE secretaria`	Crear rol
`GRANT secretaria TO ana`	Asignar un rol
$P_{\text{dados}} \subseteq P_{\text{necesarios}}$	Mínimo privilegio

26 Bases de datos NoSQL: tipos, teorema CAP y BASE

$k \mapsto v$	Clave-valor
`{nombre: 'Ana', materias: ['BD', 'Calc']}`	Documentos
$(\text{fila},\ \text{familia:col},\ t) \mapsto v$	Familia de columnas
$G = (V,\ E)$	Grafos
$\{C,\ A,\ P\} \rightarrow \text{solo 2 a la vez}$	Teorema CAP
$P \Rightarrow C \text{ o } A$	Con partición: C o A
BASE frente a ACID	BASE frente a ACID
$R + W > N$	Quórum de lectura

27 Data Warehouse: OLAP, esquema en estrella y ETL

`UPDATE matricula SET nota = 8.5 WHERE id = 7`	Operación O
$\text{HECHO}(\underline{t^*,\ e^*,\ m^*},\ \text{nota})$	Tabla de hechos
$\text{DIM_EST}(\underline{e},\ \text{nombre},\ \text{carrera})$	Tabla de dimensión
`FROM hecho h JOIN dim_est e ON h.e = e.e`	Consulta en estrella
$\text{DIM_MATERIA} \xrightarrow{\text{FK}} \text{DIM_FACULTAD}$	Copo de nieve
$E \rightarrow T \rightarrow L$	Proceso ETL
2^n	Agrupaciones del cubo
$\gamma_{\text{carrera};\ \text{AVG(nota)}}$	Roll-up